## Team 11

# Smart Bus Booking & Operations Management System
# Software Architecture Document

**Version 1.3 (Final)**

# Revision History

| Date | Version | Description | Author |
| --- | --- | --- | --- |
| 09/Nov/26 | 1.3 | Biên tập SAD thành mô tả kiến trúc độc lập; chuẩn hóa trình bày thiết kế lớp; căn chỉnh và xuất lại sơ đồ, bổ sung liên kết nguồn. Kiến trúc và các quy tắc nghiệp vụ giữ nguyên. | Đinh Đức Hiếu (tổng hợp) |

# Table of Contents


1. Introduction 4

2. Architectural Goals and Constraints 5

3. Use-Case Model 6

4. Logical View 10
   - 4.1 Component: Web Presentation 13
   - 4.2 Component: Identity & Access 16
   - 4.3 Component: Trip & Operations 19
   - 4.4 Component: Booking & Payment 25
   - 4.5 Component: AI Assistance 31

5. Deployment 34

6. Implementation View 34

# Software Architecture Document

## 1. Introduction

**Mục đích.** Tài liệu mô tả kiến trúc phần mềm của Smart Bus Booking & Operations Management System: mục tiêu và ràng buộc kiến trúc, mô hình use case, các thành phần chính cùng quan hệ giữa chúng và thiết kế lớp của từng thành phần. Tài liệu là căn cứ để nhóm hiện thực và kiểm thử từ PA4.

**Phạm vi.** Hệ thống là ứng dụng web responsive cho một nhà xe liên tỉnh, phục vụ Passenger, Staff và Administrator. Phạm vi gồm 20 use case (UC-P01–P12, UC-S01–S04, UC-A01–A04) và hành vi hỗ trợ UC-C01: tìm chuyến, đặt ghế theo Segment, thanh toán thử, vé điện tử, check-in, quản lý vận hành và hai trợ lý AI chỉ đọc. Ngoài phạm vi: thanh toán thật, bán vé nhiều nhà xe, nhắn tin/email tự động, nhắc giờ khởi hành, bắt buộc quét QR, giao diện song ngữ và chuyển hội thoại AI sang nhân viên. Mục 5 Deployment và mục 6 Implementation View được để trống cho PA4.

**Định nghĩa và thuật ngữ.**

- Route / Stop: tuyến của nhà xe, gồm danh sách điểm dừng có thứ tự.
- Trip: một chuyến chạy cụ thể của Route, có Vehicle được gán; trạng thái Scheduled → Departed → Completed, hoặc Scheduled → Cancelled.
- Segment: đoạn hành trình từ Stop lên đến Stop xuống trên một Trip; giá Segment bằng tổng giá các chặng liên tiếp.
- Seat Hold: giữ ghế tạm thời, hết hạn sau 10 phút kể từ lúc tạo thành công.
- Phân bổ hiệu lực (effective allocation): Booking Confirmed hoặc Seat Hold chưa hết hạn.
- Booking: đặt chỗ cho 1–6 ghế trên một Trip và một Segment; UC-P08 tạo Booking Pending tạm thời.
- Ticket: vé riêng cho từng ghế, chỉ phát hành sau thanh toán được máy chủ xác minh; Booking chưa xác nhận có thể chưa có Ticket.
- Reconciliation: xử lý đối soát riêng cho khoản thanh toán đến muộn, thừa hoặc không khớp.
- UC-C01 Validate Ticket for Check-in: hành vi hỗ trợ được UC-S03 bao hàm («include»).
- External AI Service (Gemini Developer API): actor hỗ trợ của UC-P11 và UC-P12 trong mô hình use case.
- FAQ đã duyệt: nội dung chính sách được phép dùng cho AI Customer Support, lưu thành bundle cấu hình có phiên bản.
- BR: quy tắc nghiệp vụ có mã số trong đặc tả use case, ví dụ BR-P11-06.
- Port: interface nội bộ giữa các module backend, ví dụ BookingQueryPort, BookingOperationsPort, TripQueryPort.

**Chữ viết tắt.** AI: Artificial Intelligence; API: Application Programming Interface; BR: Business Rule; CSRF: Cross-Site Request Forgery; DTO: Data Transfer Object; ERD: Entity-Relationship Diagram; FAQ: Frequently Asked Questions; HTTPS: Hypertext Transfer Protocol Secure; NFR: Non-Functional Requirement; RUP: Rational Unified Process; SAD: Software Architecture Document; UC: Use Case; UML: Unified Modeling Language; UUID: Universally Unique Identifier; VND: Việt Nam đồng.

**Tài liệu tham khảo.**

- Team 11, Vision Document, phiên bản 1.8, 08/11/2026.
- Team 11, PA3 Use-Case Specifications, edition 1.8 (requirements baseline 1.5); mô hình PA3_Group11_UseCaseModel_v2.2.drawio [REF-01].
- Team 11, Software Development Plan, phiên bản 1.7.
- Google AI for Developers, Gemini Developer API Pricing, Rate limits và Additional Terms of Service: tài liệu tham khảo cho cấu hình, hạn mức và xử lý dữ liệu của môi trường demo.
- Team 11, nguồn sơ đồ chỉnh sửa được: UseCaseModel v2.2, SADViews v2.3, OverallArchitecture v2.2, AIAssistance v2.3 và BookingPayment v2.1. Mỗi hình có liên kết tới đúng bộ nguồn.

- CSC13002 Nhập môn Công nghệ phần mềm, đề PA3 và mẫu rup_sad (Software Architecture Document).

**Tổng quan tài liệu.** Mục 2 nêu mục tiêu và ràng buộc kiến trúc. Mục 3 trình bày mô hình use case. Mục 4 mô tả kiến trúc tổng thể theo kiểu n-tier / modular monolith và chi tiết từng thành phần bằng sơ đồ lớp. Mục 5 và 6 để dành cho PA4.

## 2. Architectural Goals and Constraints

**Mục tiêu kiến trúc** (nguồn: Vision 1.8 mục 6)

- Toàn vẹn đặt chỗ: không có hai phân bổ hiệu lực cùng chiếm một ghế trên các Segment giao nhau của cùng Trip; hold hết hạn sau 10 phút. (NFR03)
- Độ tin cậy: sau retry, lỗi mạng, hold hết hạn, thanh toán lỗi hoặc hủy, hệ thống vẫn giữ quy tắc trên, báo lỗi rõ và không để lại phân bổ dở dang. (NFR06)
- Đúng đắn thanh toán: máy chủ xác minh kết quả; thông báo lặp xử lý idempotent; redirect trên trình duyệt không phải bằng chứng thanh toán; tối đa 3 lượt thử, một lượt Pending. (NFR05)
- Bảo mật và quyền riêng tư: xác thực, phân quyền theo vai trò, kiểm tra đầu vào, quyền tối thiểu, không lưu bí mật trong mã nguồn; câu trả lời AI về Booking phải kiểm tra quyền sở hữu. (NFR04)
- Độ tin cậy AI: AI chỉ dùng bản ghi Trip thật và nội dung chính sách đã duyệt; trả lời dự phòng khi không rõ hoặc không khả dụng; không thực hiện thao tác thay đổi dữ liệu. (NFR08)
- Hiệu năng: UC-P04 và UC-P05 đạt p95 không quá 3 giây trong benchmark lớp học (20 Route, 100 Stop, 20 Vehicle, 500 Trip, 5.000 Booking, 20 phiên đồng thời); không tính độ trễ AI và cổng thanh toán. (NFR02)
- Khả dụng: giao diện cho điện thoại rộng 360–430 px và máy tính từ 1280 px; kiểm tra trên hai phiên bản ổn định mới nhất của Chrome, Edge, Safari. (NFR01)
- Bảo trì và kiểm thử: mọi thay đổi được người khác review trước khi merge; có kiểm thử tự động cho quy tắc quan trọng; tài liệu API ghi đầu vào, đầu ra, quyền và lỗi của từng endpoint. (NFR07)

**Ràng buộc kiến trúc và công nghệ**

- Kiểu kiến trúc: n-tier / modular monolith — một backend chia module trong cùng ứng dụng, một cơ sở dữ liệu quan hệ dùng chung. Chưa dùng Redis, message broker hay microservices.
- Backend: Java 21, Spring Boot, Maven; Spring Data JPA kết hợp SQL và khóa dòng cho giao dịch quan trọng; Flyway quản lý migration.
- Frontend: React + TypeScript + Vite + Tailwind CSS; frontend và /api cùng origin qua reverse proxy.
- Cơ sở dữ liệu: PostgreSQL.
- Xác thực: Spring Security + Spring Session JDBC; cookie phiên HttpOnly, Secure khi chạy HTTPS, SameSite=Lax và CSRF token cho yêu cầu thay đổi dữ liệu. Phiên hết hạn sau 30 phút không hoạt động với Passenger, 15 phút với Staff/Administrator.
- ID nghiệp vụ dùng UUID; thời điểm lưu UTC, ngày hiển thị/lọc theo Asia/Ho_Chi_Minh. Ngày tìm chuyến tính tại Stop lên (BR-P04-05); VND lưu số nguyên và hoàn tiền làm tròn half-up một lần (BR-P10-07).
- Môi trường: prototype lớp học, chỉ dữ liệu thử, chạy bằng Docker Compose; không cam kết vận hành production hay chứng nhận thanh toán. Kết nối Sandbox và AI ghi là chưa xác minh cho đến khi có kết quả thật. (NFR09)
- Thanh toán: PA4 dùng mock có nhãn MÔ PHỎNG; PA5 dùng VNPay Sandbox. (Vision 1.8 mục 4.2)
- AI dùng Google Gemini Developer API Free Tier qua LlmGateway/GeminiGateway; model demo gemini-2.5-flash-lite. GEMINI_API_KEY/GEMINI_MODEL chỉ ở backend. Không tự chuyển sang dịch vụ trả phí; thiếu quyền truy cập, quota hoặc model thì trả fallback. Kiểm tra cấu hình thực tế trước demo.
- Phạm vi triển khai: prototype lớp học, dữ liệu giả, một nhà xe. Quy mô nhóm nhỏ và giao dịch đặt vé

liên thành phần là căn cứ chọn một backend Modular Monolith với một PostgreSQL dùng chung.

## 3. Use-Case Model

Mô hình use case kế thừa requirements baseline 1.5, trình bày trong PA3 Use-Case Specifications edition 1.8. Nguồn chỉnh sửa: PA3_Group11_UseCaseModel_v2.2.drawio, gồm tổng quan và năm view chi tiết; giữ actor, use case và quan hệ đã duyệt.

Mô hình gồm 20 use case mục tiêu người dùng — 12 của Passenger (UC-P01–UC-P12), 4 của Staff (UC-S01–UC-S04), 4 của Administrator (UC-A01–UC-A04) — và hành vi hỗ trợ UC-C01 Validate Ticket for Check-in.

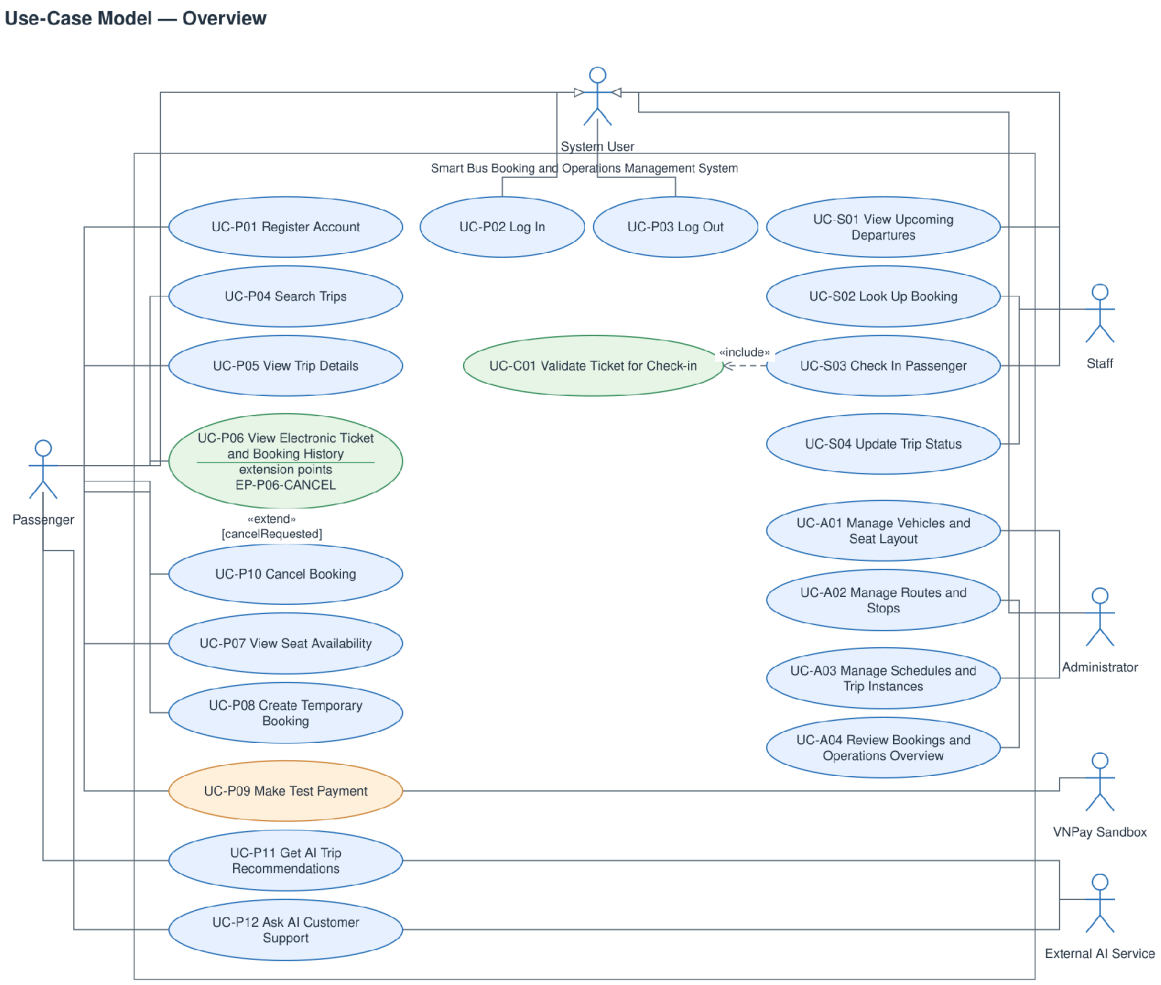
Ba actor chính là Passenger, Staff và Administrator; UC-P02 Log In và UC-P03 Log Out dùng chung cho cả ba vai trò thông qua quan hệ tổng quát hóa tới System User.

External AI Service (Gemini Developer API) là actor hỗ trợ của UC-P11 và UC-P12, Passenger vẫn là actor chính; VNPay Sandbox là actor hỗ trợ của UC-P09, dự kiến tích hợp ở PA5.

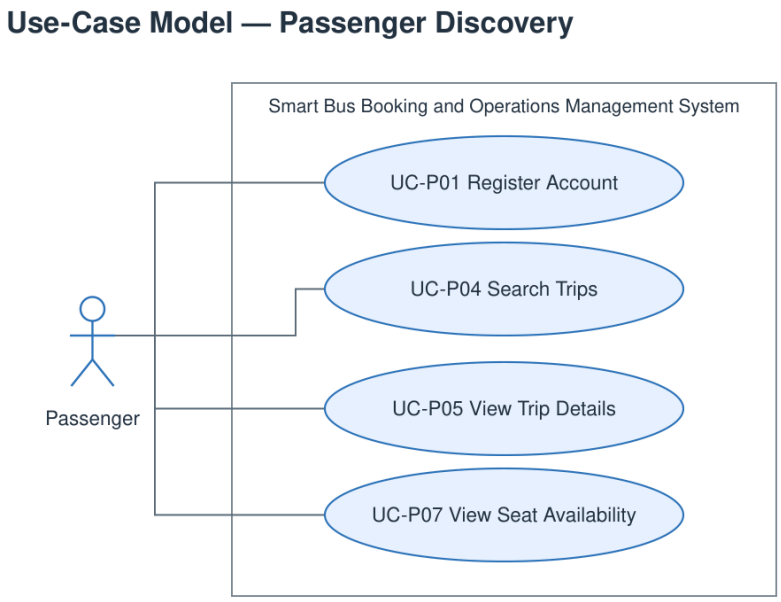
UC-S03 Check In Passenger «include» UC-C01; UC-P10 Cancel Booking «extend» UC-P06 tại điểm mở rộng EP-P06-CANCEL.

UC-P08 Create Temporary Booking kết thúc ở Booking Pending kèm giữ chỗ 10 phút; không có quan hệ «include» từ UC-P08 sang UC-P09.

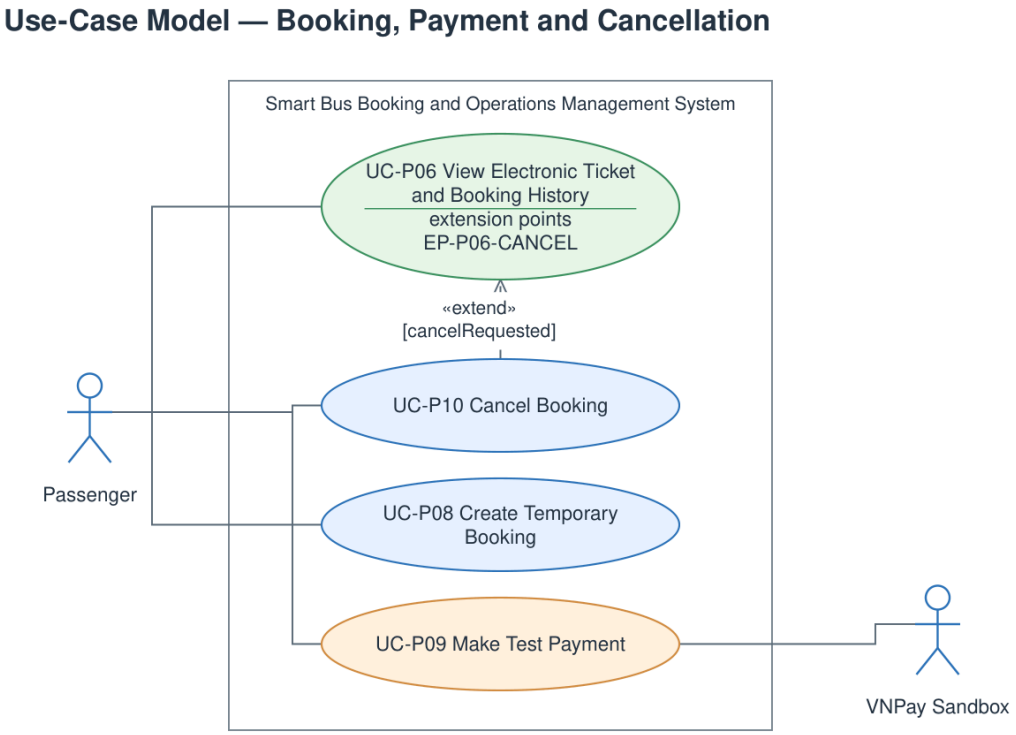
Hình 1 là sơ đồ tổng quan; Hình 2–6 là năm view chi tiết của cùng mô hình. Luồng sự kiện và quy tắc nghiệp vụ (BR) được giữ trong tài liệu đặc tả use case, không lặp lại trong SAD.



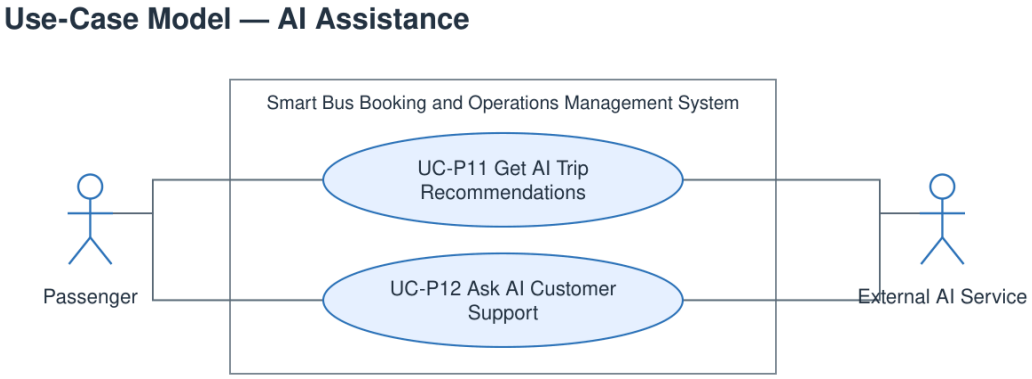

**Hình 1.** Use-Case Model — Overview *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*



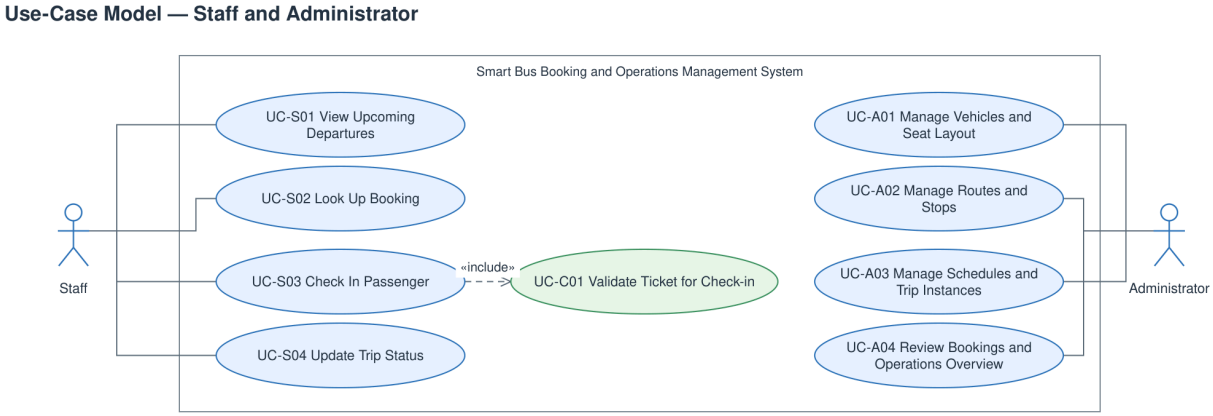

**Hình 2.** Use-Case Model — Passenger Discovery *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*



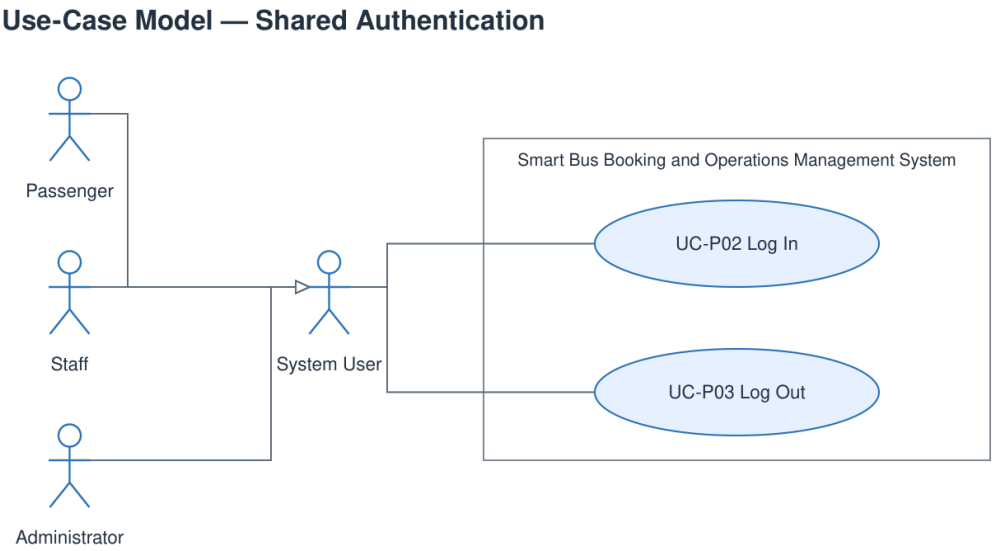

**Hình 3.** Use-Case Model — Booking, Payment and Cancellation *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*




**Hình 4.** Use-Case Model — AI Assistance *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*




**Hình 5.** Use-Case Model — Staff and Administrator *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*

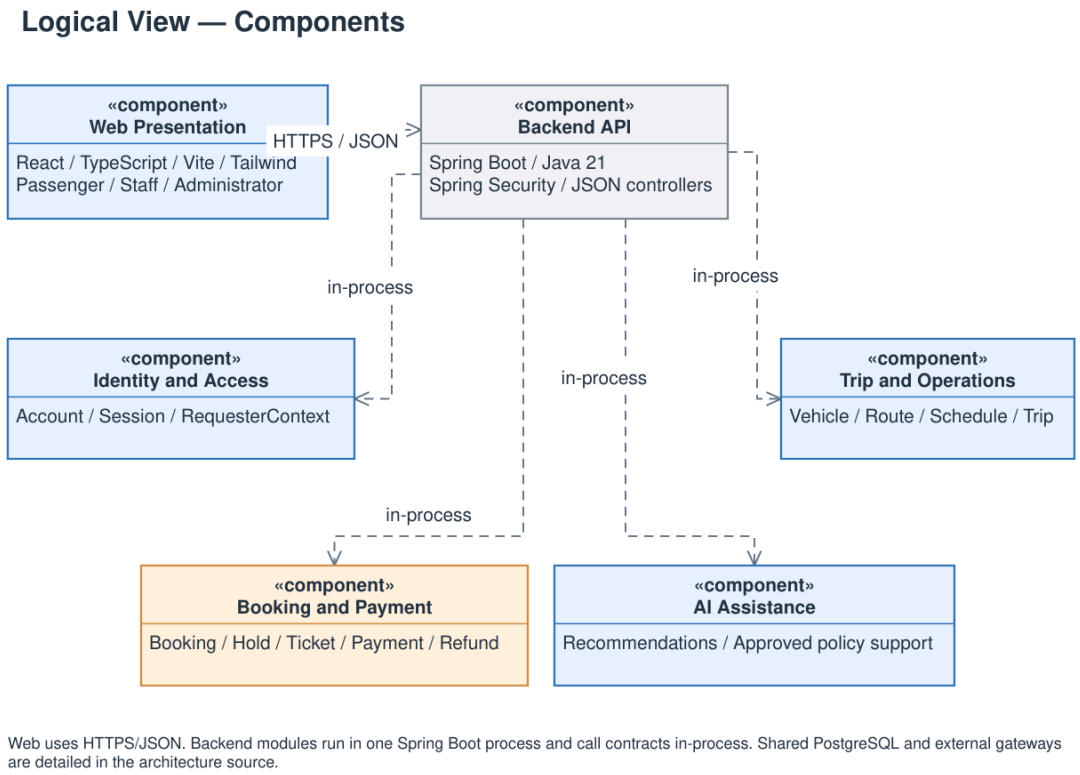



**Hình 6.** Use-Case Model — Shared Authentication *(nguồn: PA3_Group11_UseCaseModel_v2.2.drawio)*

# 4. Logical View

### *1. Các thành phần và trách nhiệm chính*

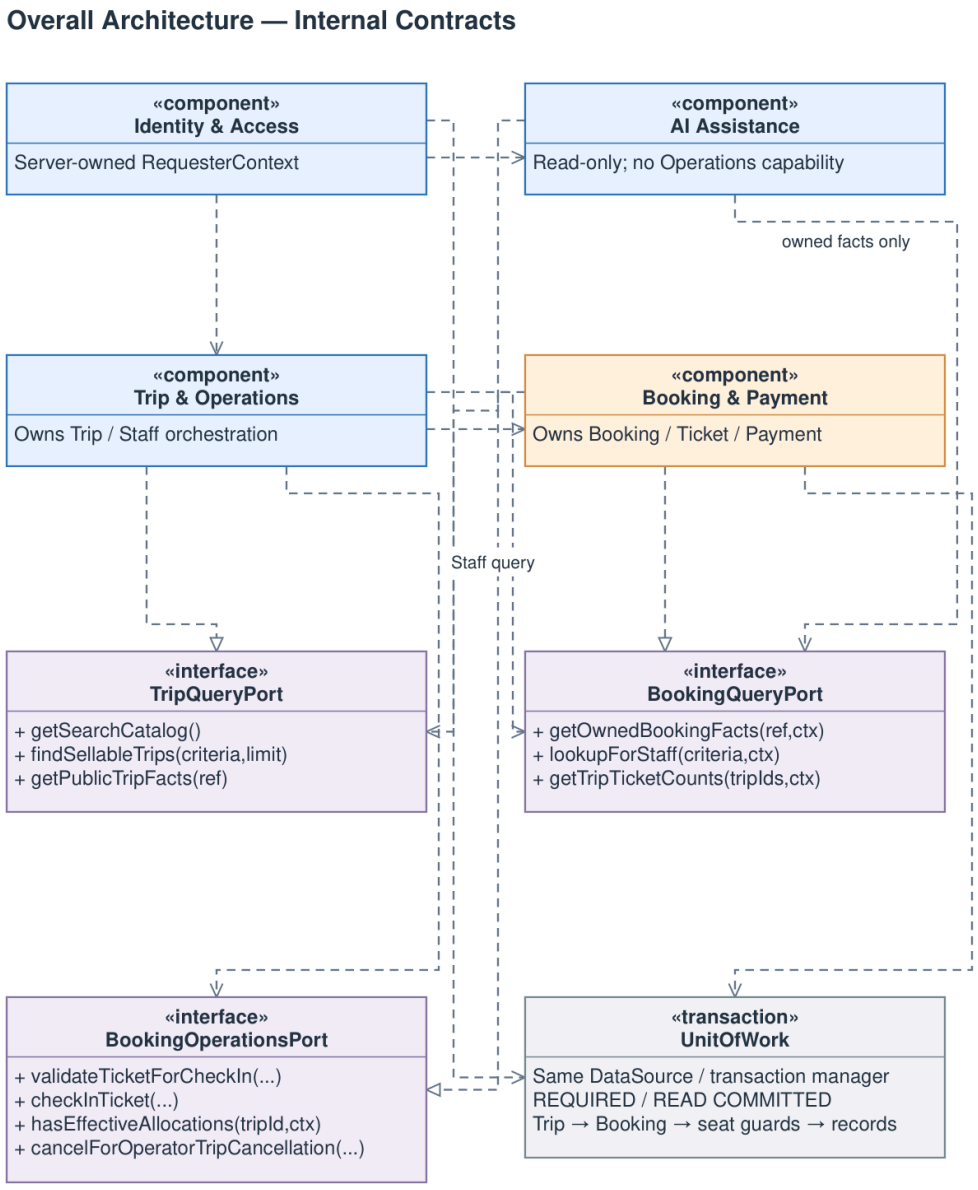
- **Web Presentation: Giao diện web** responsive (cho Passenger, Staff, Administrator); chịu trách nhiệm hiển thị dữ liệu, tiếp nhận tương tác người dùng, xử lý trạng thái rỗng/lỗi/chờ (loading) và chuyển tiếp yêu cầu đến tầng API; không tự xử lý logic nghiệp vụ hay quyền hạn ngầm phía client.
- **Identity & Access: Quản lý tài kh**oản, phiên đăng nhập (session), xác thực người dùng và phân quyền theo vai trò (Passenger, Staff, Administrator); cung cấp thông tin ngữ cảnh người gọi (RequesterContext) cho các thành phần khác.
- **Trip & Operations: Quản lý dữ liệu d**anh mục nền tảng gồm xe (Vehicle), tuyến (Route), trạm (Stop), lịch trình (Schedule) và chuyến xe (Trip); thực hiện các nghiệp vụ vận hành cốt lõi như tra cứu chuyến, check-in vé (UC-S03) và cập nhật trạng thái chuyến xe (UC-S04).
- **Booking & Payment: Quản lý luồng đặt chỗ t**heo chặng (Segment), giữ chỗ tạm thời (Hold 10 phút), vòng đời đơn hàng (Booking), lượt thanh toán (PaymentAttempt), phát hành vé điện tử (Ticket), quản lý yêu cầu hoàn tiền (Refund) và đối soát (Reconciliation).
- **AI Assistance: Hỗ trợ gợi** ý chuyến xe (UC-P11) và trả lời chăm sóc khách hàng dựa trên FAQ/dữ liệu thật (UC-P12); lọc dữ liệu cá nhân (InputSanitizer), giới hạn hạn mức (rate limit) và chỉ đóng vai trò đọc dữ liệu, không có quyền thực hiện thao tác ghi/đặt/hủy.




**Hình 7.** Logical View — Components *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

### *2. Kết nối và giao tiếp (Kết nối liên thành phần)*

- **Trình duyệt ⇄ API (Web Presentation tới các mô-đun backend):** Giao tiếp qua giao thức HTTPS định dạng JSON; sử dụng chuẩn mã lỗi, định dạng khóa camelCase và xác thực dựa trên phiên làm việc (session) phía máy chủ.
- **Giao tiếp nội bộ: gọi service/port trực tiếp** trong cùng backend. Thao tác liên module chỉ nguyên tử khi cùng DataSource, transaction manager và transaction REQUIRED; không gọi provider khi giữ khóa.
- **Tích hợp bên ngoài (Cổng thanh toán & AI):**
  - **Gemini Developer API:** HTTPS từ backend qua LlmGateway/GeminiGateway; deadline toàn yêu cầu 10 giây. Model/key lấy từ môi trường; không gọi trực tiếp từ trình duyệt.
  - **Cổng thanh toán (VNPay):** PA4 sử dụng mock có gắn nhãn mô phỏng; PA5 dự kiến tích hợp VNPay Sandbox qua HTTPS với cơ chế xác minh chữ ký bảo mật ở phía server.




**Hình 8.** Overall Architecture — Internal Contracts *(nguồn: PA3_Group11_OverallArchitecture_v2.2.drawio)*

### *3. Kiểu kiến trúc*

Backend Java 21/Spring Boot tổ chức theo Modular Monolith; React/TypeScript là giao diện; PostgreSQL/JPA-SQL/Flyway phụ trách lưu trữ. Các lớp giao diện, ứng dụng, miền và hạ tầng được phân tách trong từng module.

### *4. Cơ sở lựa chọn và ranh giới kiến trúc*

**Lý do lựa chọn:**

- Một backend giúp giảm chi phí triển khai, đơn giản hóa giao tiếp nội bộ và hỗ trợ giao dịch cục bộ cho đặt chỗ, thanh toán và vận hành.
- **Mỗi module sở hữu dữ liệu của mình: Trip & Operations g**iữ danh mục/chuyến; Booking &

Payment giữ đặt chỗ/tiền/vé; Identity & Access giữ tài khoản/phiên; AI Assistance điều phối truy vấn chỉ đọc.

- Dịch vụ ngoài nằm sau adapter: GeminiGateway cho AI và PaymentGatewayPort cho thanh toán. Domain không phụ thuộc trực tiếp vào nhà cung cấp.
  - Các lời gọi liên module dùng port có contract và quyền rõ ràng. Thay đổi liên module phải cùng transaction khi cần bảo toàn bất biến; lời gọi mạng ra provider nằm ngoài vùng giữ khóa.

## 4.1 Component: Web Presentation

### *Phạm vi và trách nhiệm*

Web responsive phục vụ Passenger, Staff và Administrator; đáp ứng trực tiếp P01–P06 và trình bày các luồng đặt vé, vận hành, quản trị, AI.

UI mô tả màn hình Booking/Payment/hủy (P07–P10), Staff (S01–S04), Admin (A01–A04) và AI (P11–P12). Web Presentation chỉ hiển thị và gọi API của module sở hữu nghiệp vụ; AI backend thuộc AI Assistance.

- UC-P04/P05 là public; UC-P06 yêu cầu phiên và dữ liệu đã được server lọc ownership.
- Giao diện không sở hữu quy tắc Trip, Seat, Booking, Payment hay Ticket; gọi contract của thành phần sở hữu.
- Không tự quyết định quyền, cutoff, availability, fare, thanh toán hoặc tính hợp lệ của vé.
- Giao diện trình bày trạng thái loading, empty, error, session expired và trạng thái nghiệp vụ theo phản hồi server. Hold countdown dựa trên expiresAt server; UI không tự gia hạn hold, phát Ticket hay kết luận thanh toán thành công từ redirect trình duyệt.
- Search gửi origin, destination và ngày tại Stop lên đã chọn; chi tiết trình bày Stop theo thứ tự, giờ, xe, fare Segment và trạng thái có thể đặt.
- Lịch sử hiển thị riêng trạng thái Booking, Payment, Ticket và Refund; không suy ra vé hợp lệ chỉ từ thanh toán thành công.
- Sau logout hoặc khi server báo session expired/revoked, xóa dữ liệu được bảo vệ khỏi UI và cache do ứng dụng kiểm soát; điều hướng UI không thay thế việc server từ chối request.
- Redirect trình duyệt không phải bằng chứng thanh toán.

### *Lớp chính và quan hệ*

**Các lớp chính:**

- **WebAppShell —** khung ứng dụng, điều hướng và vùng nội dung responsive.
- **RegistrationForm / LoginForm —** thu thập input P01/P02, hiển thị lỗi và gọi API; không xác thực có thẩm quyền.
- **SessionViewModel / RouteGuard —** phản ánh trạng thái phiên để điều hướng UI; không thay thế kiểm tra quyền server.
- **TripSearchPage / SearchCriteriaViewModel —** nhập tiêu chí và trình bày kết quả, empty/error state của P04.
- **TripDetailsPage / TripDetailsViewModel —** trình bày chi tiết và khả năng đặt do API trả về trong P05.
- **BookingHistoryPage / BookingHistoryViewModel —** tải và trình bày lịch sử P06 cùng loading/empty/error/session state.
- **BookingDetailView / TicketView —** hiển thị Booking và từng Ticket, giữ các trạng thái tách biệt; không nhân bản miền nghiệp vụ.
- **PassengerBookingFlow / StaffWorkspace / AdminWorkspace — n**hóm màn hình theo vai trò cho các UC còn lại, phụ thuộc adapter API tương ứng; không nhân bản lớp nghiệp vụ.
- **AiAssistantView / AiViewModel —** hiển thị hội thoại P11/P12, trạng thái

loading/timeout/rate-limit/fallback; không gọi model/provider trực tiếp từ browser.

- **ApiClient —** gửi HTTPS/JSON, xử lý phiên, requestId và ánh xạ lỗi theo contract chung.
- **IdentityApi / TripApi / BookingApi / AdminBookingQueryApi / AiApi «interface» —** adapter API HTTP thuộc các backend Identity, Trip & Operations, Booking & Payment và AI; frontend không gọi port nội bộ.

**Quan hệ:**

- WebAppShell chứa các Page; mỗi Page phối hợp ViewModel tương ứng; ViewModel phụ thuộc ApiClient qua IdentityApi, TripApi hoặc BookingApi.
- RouteGuard đọc SessionViewModel để hỗ trợ trải nghiệm người dùng, nhưng mọi API vẫn tự xác thực và phân quyền.
- TicketView chỉ trình bày dữ liệu từ BookingApi.

### *Mapping màn hình–API, input/output và lỗi*

Mapping dưới đây là contract thiết kế PA3; kiểm chứng endpoint/DTO khi triển khai PA4. Mọi request dùng HTTPS/JSON. Server dựng RequesterContext từ phiên; ownerId/role/staffId do client gửi không tạo quyền.

| Nhóm màn hình / UC | API operation và input chính | Output cần trình bày | Lỗi / phản ứng UI |
| --- | --- | --- | --- |
| Đăng ký P01 | IdentityApi.register(fullName,email,phone,password,passwordConfirmation) | Kết quả tạo Passenger; hướng dẫn đăng nhập, không tự tạo phiên | 422 lỗi trường; 409 email đã dùng; 503 lỗi dịch vụ; không báo thành công nếu thất bại |
| Đăng nhập/đăng xuất P02–P03 | IdentityApi.login(email,password); logout(currentSession) | Role và trạng thái phiên tối thiểu cho UI; logout xóa dữ liệu bảo vệ | 401 lỗi xác thực chung; 429 đang bị hạn chế; phiên hết hạn vẫn đóng màn hình bảo vệ |
| Tìm chuyến/chi tiết P04–P05 | TripApi.search(originStopId,destinationStopId,boardingDate); getDetails(tripId) | Danh sách chuyến, Stop theo thứ tự, giờ, xe, fare và khả năng đặt do server trả | 422 input sai; empty state khi không có chuyến; 404/503 báo lỗi phù hợp |
| Lịch sử/vé P06 | BookingApi.listOwnedBookings(criteria); backend dùng BookingQueryPort.getOwnedBookingFacts(reference, requesterContext) cho facts theo booking | Booking, Payment, Ticket, Refund/Reconciliation tách trạng thái | 401 phiên hết hạn; 403/404 không tiết lộ dữ liệu người khác; 503 không trình bày cache cũ như dữ liệu mới |
| Availability/hold/payment/hủy P07–P10 | getSeatAvailability(tripId,fromStopId,toStopId); createBooking(tripId,segment,seats,confirmedContact,idempotencyKey); createPaymentAttempt(bookingId,idempotencyKey); cancelBooking(bookingId) | Availability; Pending/hold và expiresAt; PaymentAttempt; trạng thái hủy/refund | 409 tranh ghế/conflict; HOLD_EXPIRED; 401/403; Payment Pending/unknown/Failed/late Succeeded hiển thị đúng response, không tự phát Ticket |
| Staff S01–S04 | getUpcomingDepartures(dateRange,status); lookupBooking(criteria:{bookingReference?,ticketReference?,contactPhone?}); validateTicket(tripId,ticketReference, boardingContext); checkInTicket(...); updateTripStatus(tripId,newStatus,reason) | Trang chuyến sắp chạy; Booking/Ticket facts tối thiểu; validation result (eligible/reason); check-in result/audit time; trạng thái Trip mới | 401/403; 404; 409 wrong-trip/already-checked-in/state conflict; 422 input/state transition sai; 503 retry/error. validateTicket chỉ đọc; check-in chỉ được xác nhận server-side |

| Nhóm màn hình / UC | API operation và input chính | Output cần trình bày | Lỗi / phản ứng UI |
| --- | --- | --- | --- |
| Admin A01–A04 | Vehicle/Route/Stop/Schedule/Trip API nhận criteria và dữ liệu form; A04 getOperationsOverview(filters,page) chỉ đọc | Danh sách/chi tiết theo response; lỗi validation theo trường; A04 KPI và trang kết quả theo bộ lọc | 401/403; 404; 409 duplicate/conflict; 422 input/bộ lọc sai; 503 retry/error. A04 rỗng hiển thị 0; không có thao tác sửa Booking/duyệt tiền/refund |
| AI P11–P12 | AiApi.recommendTrips(input: AiRequest) / askSupport(input: AiRequest); adapter chuyển tới AiAssistantApi cùng tên operation. Context và hội thoại ẩn danh do server xác lập. | Đề xuất/câu trả lời; không tự sửa Booking/Payment | 401 khi yêu cầu dữ liệu riêng tư; 429; timeout/503 có fallback; không trả Booking nếu chưa xác thực/ownership |

BookingQueryPort và BookingOperationsPort là port nội bộ backend, không phải API trình duyệt. Adapter gọi operation với RequesterContext và kiểm tra quyền; AI chỉ được dùng getOwnedBookingFacts qua kiểm tra ownership. Staff dùng đúng quyền vận hành, A04 chỉ đọc; AI ẩn danh chỉ truy vấn dữ liệu công khai/FAQ.

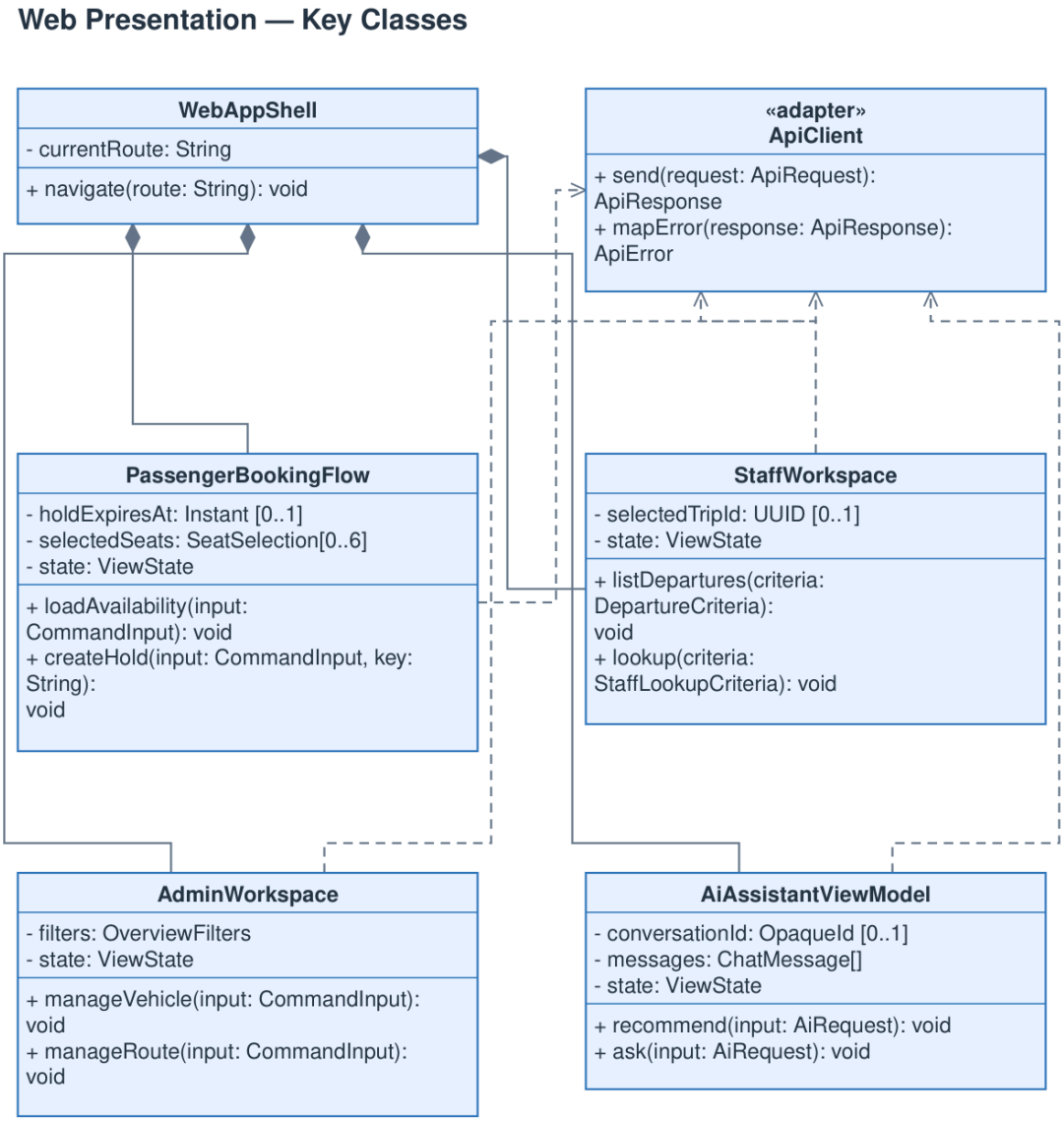
### *Trạng thái UI*

Mọi luồng bất đồng bộ có loading/empty/error và retry phù hợp. UI thể hiện hold hết hạn, tranh ghế, payment pending/unknown/failed/late-succeeded, vé đã hủy, Staff wrong-trip/already-checked-in, Admin overview rỗng/lỗi và AI timeout/429/fallback.

## *3. Kết nối, giao tiếp và công nghệ*

- Trình duyệt giao tiếp backend bằng HTTPS với JSON theo API contract; không truy cập trực tiếp DB hoặc gọi lớp nghiệp vụ nội bộ.
- API HTTP dùng tiền tố /api/v1; session cookie và CSRF bảo vệ các thao tác browser. Endpoint/DTO được kiểm chứng qua integration test khi hiện thực.
- Vision xác định responsive web cho Chrome, Edge, Safari trên máy tính và thiết bị di động; viewport kiểm tra 360–430 px và từ 1280 px.
- Passenger chủ yếu dùng điện thoại; Staff dùng điện thoại/workstation; Administrator dùng desktop/laptop.
- Web Presentation dùng React/TypeScript/Vite/Tailwind. Frontend và /api cùng origin; ApiClient gửi cookie phiên và CSRF token cho thao tác thay đổi dữ liệu.
- Sơ đồ lớp mô tả quan hệ giữa Page, ViewModel và adapter API; mỗi tên lớp tham chiếu một trách nhiệm nhất quán.
- UI cần đáp ứng NFR01 (không tràn ngang/cắt controls; trạng thái thành công/lỗi khớp UC) và hỗ trợ đo NFR02 cho P04/P05: p95 không quá 3 giây trong benchmark Vision quy định.

### *4. Sơ đồ lớp*




**Hình 9.** Web Presentation — Key Classes *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

## 4.2 Component: Identity & Access

### *Phạm vi và trách nhiệm*

- Thành phần sở hữu danh tính tài khoản, xác thực dùng chung, vòng đời phiên và quyết định quyền truy cập.
- Trực tiếp đáp ứng UC-P01 Register Account, UC-P02 Log In và UC-P03 Log Out; cung cấp ngữ cảnh xác thực và kiểm tra quyền/ownership cho UC-P06.
- UC-P04 Search Trips và UC-P05 View Trip Details là chức năng công khai.

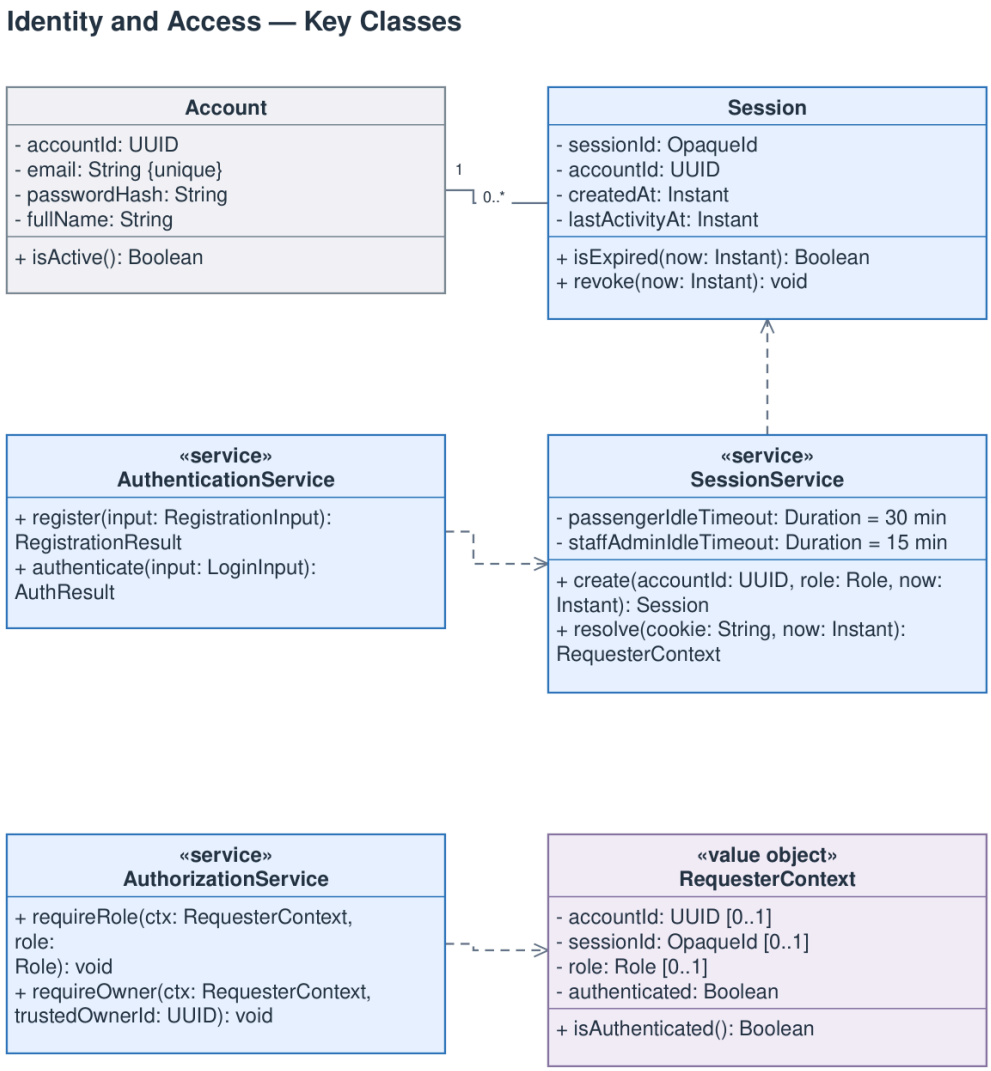
- Đăng ký tạo Passenger với email duy nhất, kiểm tra trường bắt buộc và mật khẩu xác nhận; không tự đăng nhập.
- Đăng nhập xác minh thông tin và trạng thái account, trả thông báo lỗi chung, chỉ tạo phiên sau khi thành công.
- LoginAttemptTracker đếm các lần sai liên tiếp trong cửa sổ trượt 15 phút. Đạt 5 lần thì hạn chế 5 phút; hết hạn được thử lại. Đăng nhập thành công xóa bộ đếm và thời điểm hạn chế.
- Server áp dụng sliding idle timeout: 30 phút cho Passenger, 15 phút cho Staff/Administrator. Chỉ request tới API được bảo vệ đã xác thực, qua kiểm tra quyền và được server chấp nhận mới cập nhật lastActivityAt; thao tác UI cục bộ, request công khai hoặc request thất bại không gia hạn phiên. Server tính expiresAt từ lastActivityAt và timeout theo role. resolve() từ chối session expired/revoked; recordActivity() không thể phục hồi chúng. Logout (P03) phải revoke session ở server; mọi request tiếp theo dùng session đó bị từ chối, còn Web Presentation xóa dữ liệu bảo vệ khỏi UI và cache do ứng dụng kiểm soát. Cơ chế đã chốt là session cookie; không có refresh-token flow.
- Mọi API bảo vệ phải kiểm tra phiên, role và ownership phía server.
- UC-P06 chỉ trả dữ liệu Booking/Ticket thuộc Passenger hiện tại.

### *Lớp chính và quan hệ*

**Các lớp chính:**

- **Account —** accountId, email (unique), passwordHash, fullName, phone, status; biểu diễn một danh tính người dùng.
- **Role «enumeration» —** PASSENGER, STAFF, ADMINISTRATOR; role được gán qua quy trình provision ngoài phạm vi UC công khai.
- **AuthenticationService —** register(), authenticate(); điều phối P01/P02 và chỉ phát hành phiên sau khi xác thực thành công.
- **CredentialPolicy —** validateRegistration(), verifyPassword(); kiểm tra dữ liệu đăng ký và thông tin đăng nhập.
- **LoginAttemptTracker —** failedAt: DateTime[*] (chỉ các lần sai trong 15 phút gần nhất), restrictedUntil: DateTime [0..1]; recordFailure(loginKey: String, now: DateTime): void, isRestricted(loginKey: String, now: DateTime): Boolean, resetOnSuccess(loginKey: String): void. Đạt 5 lần sai liên tiếp trong cửa sổ trượt 15 phút thì đặt restrictedUntil = now + 5 phút; sau thời điểm này có thể thử lại, không khóa vĩnh viễn; xác thực thành công xóa bộ đếm và hạn chế.
- **Session —** sessionId, accountId, createdAt, lastActivityAt, expiresAt, revokedAt; biểu diễn phiên có thời hạn.
- **SessionService —** create(accountId,role,now), resolve(sessionId,now), recordActivity(sessionId,acceptedProtectedRequest,now), revoke(sessionId,now); quản lý phiên và logout P03.
- **AuthorizationService —** requireRole(), requireOwner(); kiểm tra quyền theo RequesterContext và contract ownership của Booking.
- **RequesterContext c**hứa accountId, sessionId, role khi authenticated=true; các trường này có thể vắng khi authenticated=false. AI ẩn danh dùng anonymousConversationId do server cấp, độc lập với Session đã xác thực và không có quyền ownership.
- **AccountRepository lưu Account trong Post**greSQL; SessionRepository dùng Spring Session JDBC. Session logical ánh xạ sang bảng hạ tầng của Spring Session.
- **Quan hệ:** Một Account có thể có 0 hoặc nhiều Session; mỗi Session thuộc đúng một Account.
- Account gắn một Role.
- AuthenticationService phụ thuộc CredentialPolicy, LoginAttemptTracker, AccountRepository và SessionService.
- AuthorizationService nhận RequesterContext.

Identity cung cấp danh tính và quyền cho các module, không sở hữu Booking/Ticket. Định danh hội thoại ẩn danh độc lập với phiên đã xác thực; nó không chứng minh role hoặc ownership và không được dùng để đọc Booking riêng tư.




**Hình 10.** Identity and Access — Key Classes *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

### *Cách thức kết nối*

- Web Presentation gọi API Identity qua HTTPS/JSON.
- Server xác thực phiên ở mỗi yêu cầu, dựng RequesterContext và kiểm tra quyền trước khi trả dữ liệu hoặc thực hiện thao tác.
- Identity & Access dùng Spring Security, PasswordEncoder và Spring Session JDBC. Phiên được lưu phía server, bảo vệ bằng cookie và có thể thu hồi khi logout.

- Mật khẩu dùng bộ mã hóa chuyên dụng có salt qua Spring Security PasswordEncoder; không lưu bản rõ hoặc ghi credential/session secret vào log.
- Bảo mật cần được kiểm chứng bằng kiểm thử xác thực, phân quyền, ownership và vòng đời phiên trong quá trình hiện thực.

## 4.3 Component: Trip & Operations

### *1. Mục đích, phạm vi và ánh xạ các ca sử dụng (Use Case)*

Thành phần Trip & Operations chịu trách nhiệm quản lý toàn bộ dữ liệu danh mục cốt lõi của nhà xe và hỗ trợ các tác vụ vận hành hằng ngày của nhân viên. Thành phần này đáp ứng 7 Use Case chính:

- **Nghiệp vụ Administrator:** UC-A01 (Quản lý xe và sơ đồ ghế), UC-A02 (Quản lý tuyến đường và trạm dừng), UC-A03 (Quản lý lịch trình và khởi tạo chuyến xe).
- **Nghiệp vụ Staff:** UC-S01 (Xem danh sách khởi hành sắp tới), UC-S02 (Tra cứu đặt chỗ - phối hợp giao diện/truy vấn vé từ thành phần Booking), UC-S03 (Check-in hành khách), UC-S04 (Cập nhật trạng thái chuyến xe).

### *2. Trách nhiệm chính của thành phần*

- **Quản lý dữ liệu nền tảng:** Lưu trữ và quản lý vòng đời của Vehicle (có layout ghế ngồi/giường nằm cố định), Route (danh sách Stop có thứ tự), Schedule và Trip (kèm thời gian dừng và giá vé từng chặng LegFare).
- **Bảo vệ toàn vẹn dữ liệu:** Ngăn chặn việc thay đổi cấu trúc sơ đồ ghế hoặc sức chứa của xe khi đã được gán cho bất kỳ chuyến xe nào. Không cho phép thay đổi tuyến đường, thời gian hoặc giá vé của chuyến xe khi đã có Booking Confirmed hoặc Seat Hold còn hiệu lực.
- **Check-in theo Ticket qua Booking & Pa**yment; Trip Scheduled hoặc Departed có điểm đón trung gian hợp lệ. Vòng đời Trip: Scheduled → Departed → Completed; chỉ Scheduled → Cancelled.
- **Trip & Operations** sở hữu ghế vật lý và layout; Booking, BookingSeat, Ticket và Payment thuộc Booking & Payment.

### *3. Hợp đồng dữ liệu và kết nối với các thành phần khác*

- **Với Booking & Payment: Cung cấp tr**ipId, trạng thái Trip, danh sách Stop có thứ tự, giờ xuất phát, layout/ghế vật lý và giá từng chặng (Segment) để phục vụ việc giữ ghế và thanh toán. Nhận lại các thông tin Ticket, Booking để thực hiện tác vụ tra cứu (UC-S02) và Check-in (UC-S03).
- **Với AI Assistance: Cung cấp gi**ao diện TripQueryPort (chỉ đọc) để AI truy xuất danh mục trạm, loại xe và tìm kiếm các chuyến xe (Scheduled, còn trước mốc đóng bán 30 phút).
- **Với Identity & Access: Nhận thô**ng tin xác thực (sessionId, role) để kiểm tra quyền truy cập của Staff và Administrator trước khi thực hiện các thao tác thay đổi dữ liệu hoặc check-in.
- **Với Web Presentation: Cung cấp** dữ liệu để hiển thị giao diện danh sách chuyến xe, xe, tuyến, trạm, sơ đồ ghế và thông báo lỗi liên quan đến nghiệp vụ vận hành.

### *4. KIẾN TRÚC TỔNG THỂ VÀ LUỒNG ĐIỀU HƯỚNG API (OVERALL ARCHITECTURE & API ROUTING)*

#### 4.1. Phân luồng API bên ngoài và Giao tiếp dịch vụ nội bộ

Kiến trúc tổng thể của Smart Bus Booking & Operations Management System trình bày ở Logical View. Phần này cụ thể hóa đầu vào HTTP và contract Trip & Operations:

**1. Giao tiếp bên ngoài (External Client-to-Backend Entry Points):**

Toàn bộ truy cập từ phía người dùng trên Web Presentation (React + TypeScript) trên trình duyệt máy tính/di động đều thông qua giao thức an toàn HTTPS / RESTful JSON API. Các Controller đóng vai trò là điểm vào (API Entry Points) duy nhất cho từng phân hệ backend tương ứng:

- **Identity & Access Controller (/api/v1/auth/**, /api/v1/users/**):** đăng ký, đăng nhập/đăng xuất và

kiểm tra phiên cookie phía server; CSRF cho thao tác browser thay đổi dữ liệu.

- **Trip & Operations Controller (/api/v1/trips/**, /api/v1/schedules/**, /api/v1/routes/**, /api/v1/vehicles/**):** Tiếp nhận luồng tra cứu lịch trình, khởi tạo/cập nhật chuyến xe, danh mục xe/tuyến và các thao tác check-in hành khách.
- **Booking & Payment Controller (/api/v1/bookings/**, /api/v1/tickets/**, /api/v1/payments/**):** Tiếp nhận luồng đặt vé theo chặng (Segment), giữ chỗ tạm thời (Seat Hold), thanh toán qua cổng ngoại vi và xử lý yêu cầu hoàn tiền.
- **AI Assistance Controller (/api/v1/ai/**):** Tiếp nhận các truy vấn tư vấn lịch trình thông minh và hỗ trợ tra cứu tự động cho người dùng.

**2. Giao tiếp nội bộ giữa các phân hệ (Internal Inter-Module Communication):**

Các phân hệ backend nằm chung trong một ứng dụng duy nhất (In-Process Execution) và giao tiếp với nhau trực tiếp bằng lời gọi dịch vụ nội bộ (Internal Method Calls) thông qua các Interface Ports công khai (BookingQueryPort, BookingOperationsPort, TripQueryPort).

Các lời gọi qua port chạy nội bộ; tính nguyên tử chỉ được bảo đảm khi cùng DataSource/transaction manager và transaction REQUIRED, không tự có chỉ vì nằm chung tiến trình.

### 4.2. Căn cứ và Lý do lựa chọn kiến trúc Modular Monolith

Kiến trúc Backend Modular Monolith được nhóm thống nhất lựa chọn dựa trên 3 căn cứ thực tiễn:

- Phù hợp quy mô năm thành viên:
  - Tối ưu hóa chi phí vận hành và hạ tầng (DevOps Overhead). Nhóm không phải tốn thời gian xây dựng và quản lý các thành phần phân tán phức tạp như API Gateway, Service Discovery (Eureka/Consul), Circuit Breaker hay Distributed Tracing (Zipkin/Jaeger) như ở kiến trúc Microservices.
- Đảm bảo tính vẹn toàn giao dịch đặt vé (Booking ACID Transactions):
  - Nghiệp vụ xe khách đòi hỏi tính nhất quán cao trong các kịch bản cạnh tranh khốc liệt: Giữ ghế tạm thời P08, Check-in tại bến S03 và Hủy chuyến nhà xe S04.
  - Dùng chung một cơ sở dữ liệu quan hệ cho phép hệ thống sử dụng Giao dịch nội bộ (Local Transaction) kết hợp Khóa bi quan (SELECT ... FOR UPDATE), hỗ trợ tính nhất quán khi kiểm tra/ghi cùng transaction và cơ chế khóa/retry đúng.
- Tối ưu tốc độ phát triển và kiểm thử (Development Velocity & CI/CD):
  - Chia sẻ contract/DTO công khai giữa các module; mỗi module giữ quyền ghi entity của mình, giúp các thành viên dễ dàng refactor mã nguồn.
  - Tiến trình tích hợp liên tục (CI/CD) và kiểm thử tự động (Integration Testing) diễn ra nhanh chóng trên cùng một đơn vị đóng gói duy nhất (.jar / .war).

### 4.3. Minh bạch trạng thái công nghệ sử dụng

Công nghệ dưới đây mô tả thiết kế hệ thống. Các kết quả triển khai và kiểm thử được ghi nhận riêng khi thực hiện.

- Công nghệ đã chốt chính thức (Approved):
  - **Frontend:** React + TypeScript + Vite + Tailwind CSS.
  - **Backend:** Java 21 / Spring Boot / Maven, Modular Monolith.
  - CSDL PostgreSQL; Spring Data JPA và SQL/khóa dòng; Flyway quản lý migration. ERD logical v2.0 có 23 thực thể, số bảng vật lý theo mapping/migration; không cố định 18 bảng.
  - **Kiểu dữ liệu & Chuẩn mã hóa:** Tiền tệ lưu số nguyên (bigint - VND); Thời gian lưu định dạng ISO-8601 UTC và hiển thị theo Asia/Ho_Chi_Minh.
- Cơ chế xác thực, phiên và môi trường chạy:
  - Session lưu PostgreSQL qua Spring Session JDBC; chưa đưa Redis vào phạm vi ban đầu.

- **Xác thực:** Spring Security, session cookie HttpOnly/Secure/SameSite=Lax và CSRF; thu hồi phiên tại server; không dùng JWT/refresh-token trong phương án hiện hành.
- **Thanh toán:** PA4 mock có nhãn mô phỏng; PA5 VNPay Sandbox theo baseline, không thêm ZaloPay.

### *5. Chính sách vòng đời giữa Schedule và Trip*

- Quan hệ giữa Schedule và Trip là quan hệ liên kết khởi tạo (Association - generates), không phải quan hệ sở hữu sinh tử (Composition).
- Schedule đóng vai trò là khuôn mẫu (template) lập lịch.
- Trip là thực thể chuyến xe cụ thể độc lập, lưu trữ lịch trình thực tế, danh sách ghế, lịch sử check-in và các đơn đặt vé của hành khách.
- Khi Admin hủy hoặc ngừng kích hoạt một Schedule, các Trip đã được khởi tạo trước đó vẫn được giữ nguyên độc lập trong CSDL để bảo toàn dữ liệu lịch sử vận hành và vé đã bán.

### *6. Khóa, ràng buộc toàn vẹn và cơ chế tính giá chặng (LegFare)*

Để bảo đảm tính hợp lệ CSDL và hỗ trợ mô-đun Booking tính giá vé chặng (Segment) chính xác, hệ thống áp dụng các quy tắc khóa và ràng buộc tham chiếu sau:

**Ràng buộc khóa & chỉ số duy nhất (Database Constraints):**

- **ROUTE_STOP:** PK(route_id,stop_id), UK(route_id,stop_order); stopOrder tăng dần, duy nhất trong Route; một Stop không lặp trong cùng Route.
- **TRIP_STOP:** Khóa chính ghép PK(trip_id, stop_id). Danh sách các stop_id trong TRIP_STOP phải thuộc đúng tập hợp trạm của ROUTE được phân công cho Trip đó.
- **LEG_FARE:** Khóa chính ghép PK(trip_id, from_stop_id, to_stop_id). Ràng buộc giá trị không âm CHECK (fare_amount_vnd >= 0).

**Cơ chế liên kết thứ tự & tính giá chặng (Segment Fare Rule):**

LEG_FARE có PK(trip_id,from_stop_id,to_stop_id), hai FK ghép tới TRIP_STOP của cùng Trip, fare_amount_vnd >= 0; Stop đích ngay sau Stop nguồn. Mỗi cặp Stop liên tiếp có một LegFare; segmentFareVnd = SUM(fare_amount_vnd) cho fromStopIndex <= legIndex < toStopIndex.

Với hai Stop có chỉ số i < j, giá Segment là tổng LegFare của các cặp liên tiếp từ i đến j−1; không dùng chênh lệch thời gian để suy giá.

Mô-đun Booking của Booking & Payment sẽ truy vấn danh sách LEG_FARE này từ mô-đun Trip để làm căn cứ xác định giá tiền và tạo đơn hàng.

### *7. Ranh giới Transaction & Cơ chế kiểm soát tranh chấp (Concurrency & Transaction Boundaries)*

Kiểm soát tranh chấp bằng khóa bi quan và kiểm tra điều kiện dưới khóa trong cùng transaction PostgreSQL; khi lỗi rollback toàn bộ. Đây là thiết kế, cần integration test PA4.

#### 7.1. Nguyên tắc chung cho mọi Đơn vị công việc (Unit of Work)

- **Một transaction REQUIRED, cùng DataSource và transa**ction manager, isolation READ COMMITTED. Không tách REQUIRES_NEW cho một phần cập nhật liên module.
- **Thứ tự khóa: Trip → Booking tăng dần → gua**rd(tripId,physicalSeatId) tăng dần → Ticket/PaymentAttempt/PaymentTransaction → RefundRequest/ReconciliationCase. Dùng SELECT ... FOR UPDATE tại phần sở hữu dữ liệu.
- **Re-check nguyên tử trước khi Commit:**Ngay trước lệnh COMMIT, hệ thống bắt buộc thực hiện kiểm tra lại (re-verify) các điều kiện ràng buộc phân bổ ghế và trạng thái vé. Nếu phát hiện dữ liệu bị thay đổi bởi tiến trình song song trong khoảng thời gian xử lý, toàn bộ Transaction sẽ bị ROLLBACK lập tức và trả về thông báo lỗi thích hợp.

**7.2. Giải pháp kiểm soát tranh chấp cho 3 kịch bản trọng yếu**

**1. Tranh chấp giữa UC-A03 (Admin sửa Trip) và P08 (Hành khách giữ ghế)**

Nguy cơ: Admin sửa Trip đồng thời Passenger tạo hold trên web responsive.

Quy trình xử lý giao dịch:

- Khi Admin gửi yêu cầu lưu chỉnh sửa Trip (UC-A03), hệ thống mở Transaction và phát lệnh:SELECT * FROM trip WHERE trip_id = ? FOR UPDATE;
- BookingOperationsPort.hasEffectiveAllocations(tripId,ctx): AllocationGuardResult; ctx phải là Admin hợp lệ của A03. Gọi dưới khóa Trip trong cùng transaction.
- Sau khi khóa Trip, kiểm tra allocation trước khi lưu. Có Confirmed allocation hoặc hold còn hạn thì rollback và báo xung đột. P08 cũng khóa Trip nên không thể chèn hold vượt qua cùng vùng kiểm tra.
- Tiến trình P08 của hành khách khi cố gắng tạo Seat Hold cũng sẽ bị chặn ở bước khóa dòng cho đến khi Transaction của UC-A03 kết thúc.

**2. Tranh chấp UC-S03 check-in và UC-P10 hủy Booking**

Nguy cơ: Staff xác nhận check-in đúng lúc Passenger hủy toàn Booking trên web responsive.

Quy trình xử lý giao dịch:

- UC-S03 xác định Trip/Booking từ Ticket; khóa Trip → Booking → guard ghế → Ticket trong cùng transaction. BookingOperationsPort.checkInTicket tái kiểm tra quyền và điều kiện, ghi audit sau xác nhận của Staff. Trip & Operations không ghi Ticket ngoài dịch vụ của Booking & Payment.
- validateTicketForCheckIn(tripId,ticketReference,boardingContext,ctx): TicketValidationResult chỉ đọc; checkInTicket(cùng tham số): CheckInResult phải kiểm tra lại dưới khóa và sau xác nhận của Staff.
- Nếu tiến trình P10 (Hủy Booking) đang chạy song song, một trong hai tiến trình sẽ lấy được khóa trước, tiến trình còn lại phải chờ ở trạng thái khóa.
- Kịch bản P10 hoàn tất trước: Vé chuyển sang trạng thái CANCELLED. Sau khi lấy được khóa, tiến trình UC-S03 re-check thấy vé đã hủy ROLLBACK Transaction và từ chối check-in.
- Kịch bản UC-S03 hoàn tất trước: Vé chuyển sang trạng thái CHECKED_IN. Khi tiến trình P10 lấy được khóa và re-check thấy vé đã CHECKED_IN ROLLBACK và từ chối yêu cầu hủy vé của hành khách.

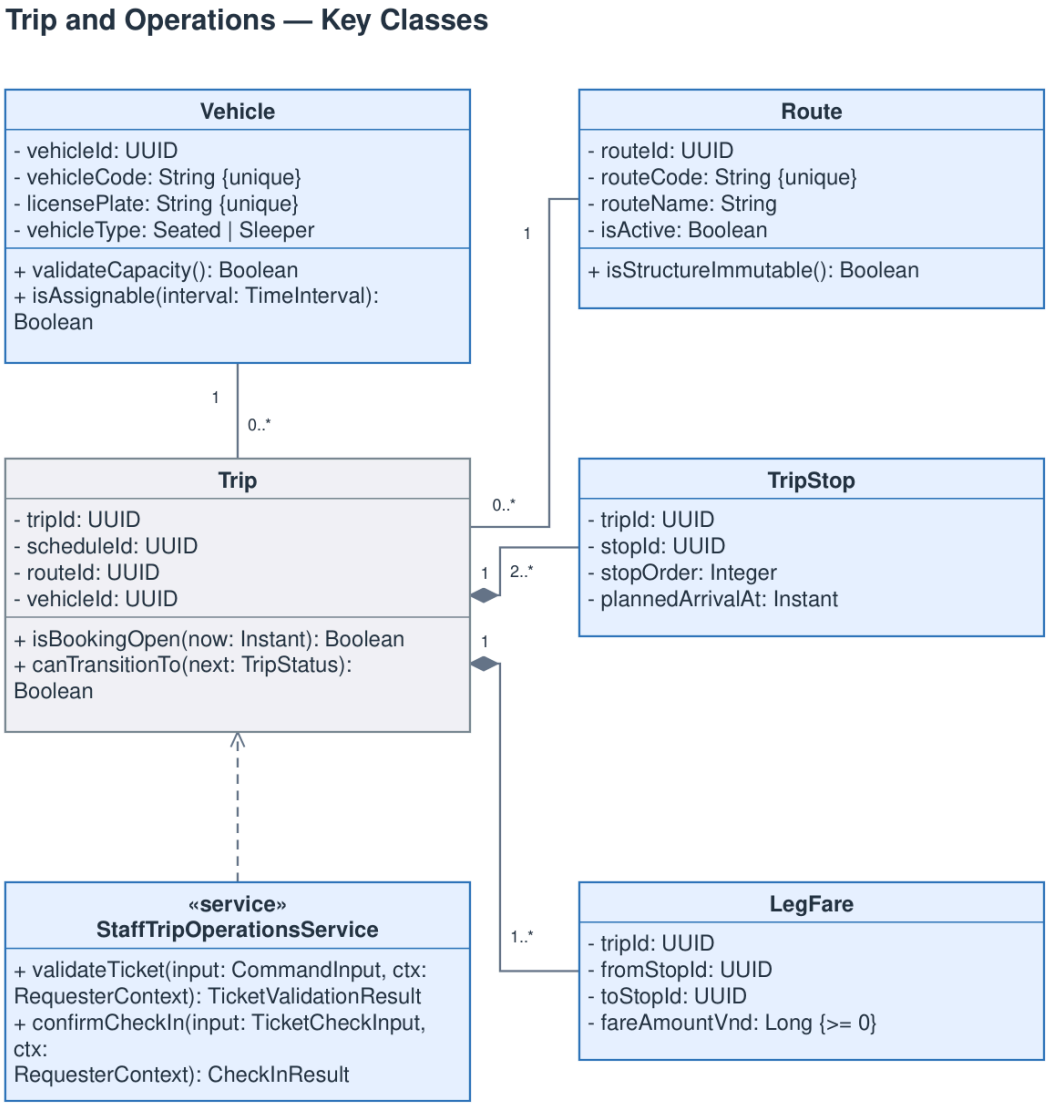
**3. Tranh chấp giữa UC-S04 (Nhà xe hủy Trip) và P08/P09 (Giữ ghế / Thanh toán)**

Nguy cơ tranh chấp: Staff thao tác hủy toàn bộ chuyến xe (UC-S04) trong lúc hành khách đang thực hiện giữ ghế (P08) hoặc tiến hành thanh toán (P09).

Quy trình xử lý giao dịch:

- UC-S04 mở Transaction, khóa bản ghi chuyến xe:SELECT * FROM trip WHERE trip_id = ? FOR UPDATE;và cập nhật status của chuyến xe thành CANCELLED.
- Trong cùng transaction: cancelForOperatorTripCancellation(tripId,reason,ctx): OperatorCancellationResult; hủy Booking/Ticket, giải phóng allocation và ghi nghĩa vụ hoàn 100% có đối chiếu nguồn tiền; lỗi rollback cả Trip.
- P08/P09 sau khi lấy khóa phải đọc lại Trip Cancelled và không tạo hold/Confirm/Ticket. Khoản tiền đã xác minh đến muộn vẫn ghi nhận, đối soát và ghi nghĩa vụ hoàn, không bỏ chứng cứ tiền.

***Thiết kế lớp và mô hình dữ liệu hợp nhất***




**Hình 11.** Trip and Operations — Key Classes *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

**1. Các lớp chính và thuộc tính**

Dựa trên đặc tả UC-A01 và UC-A03 của baseline 1.5:

- **Vehicle (Xe):** vehicleId, vehicleCode, licensePlate (Biển số duy nhất), vehicleType (seated/sleeper), capacity (Sức chứa), layoutTemplate (Mẫu sơ đồ cố định), isActive (Cho phép deactivation thay vì xóa).
- **PhysicalSeat (Ghế vật lý):** physicalSeatId, vehicleId, seatCode (Mã ghế, số lượng phải bằng capacity). Giải thích: Tách danh sách ghế vật lý để thành phần Booking của Booking & Payment có thể tham chiếu ID khi đặt vé mà không phụ thuộc vào chuỗi JSON của layout.

- **Stop (Trạm dừng):** stopId, stopName, address, isActive.
- **Route (Tuyến đường):** routeId, routeCode, routeName, isActive.
- **RouteStop (Trạm thuộc tuyến):** routeId, stopId, stopOrder (Thứ tự trạm dừng).
- **Schedule (Lịch trình):** scheduleId, routeId, operatingDays (Ngày hoạt động), firstStopDepartureTime (Giờ xuất phát tại trạm đầu).
- **Trip (Chuyến xe cụ thể):** tripId, scheduleId, routeId, vehicleId, departureDate (Ngày chạy), status (Scheduled, Departed, Completed, Cancelled).
- **TripStop (Thời gian tại trạm của chuyến):** tripId, stopId, plannedArrivalTime, plannedDepartureTime (Bắt buộc phải tăng dần theo stopOrder).
- **LegFare: tripId,fromStop**Id,toStopId,fareAmountVnd; PK(tripId,fromStopId,toStopId), FK ghép về TripStop. Hai Stop phải liên tiếp; index chỉ phục vụ thứ tự và tính tổng giá Segment.

**2. Quan hệ và Ownership (Quyền sở hữu)**

**2.1. Phân định quyền sở hữu thực thể (Entity Ownership)**

- **Trip & Operations sở hữu Vehicle, PhysicalSeat, Sto**p, Route, RouteStop, Schedule, Trip, TripStop và LegFare.
- **Booking & Payment sở hữu Booking, BookingSeat, SeatAlloca**tion, PaymentAttempt, PaymentTransaction, Ticket, RefundRequest và ReconciliationCase.
- **Identity & Access sở hữu Account và Session; Role là** enumeration PASSENGER/STAFF/ADMINISTRATOR.

**2.2. Phân biệt ID tham chiếu và Snapshot dữ liệu nghiệp vụ (Snapshot Policy)**

Để giải quyết triệt để ranh giới giữa việc tránh nhân bản mô hình dữ liệu (Data Duplication) và yêu cầu bảo toàn lịch sử giao dịch tuyệt đối khi thông tin danh mục thay đổi, hệ thống quy định rõ hai cơ chế lưu trữ song song:

**1. Khóa ngoại tham chiếu động (Dynamic Reference Foreign Keys):**

- Mô-đun Booking & Payment lưu các ID tham chiếu (trip_id, physical_seat_id, from_stop_id, to_stop_id) tới mô-đun Trip & Operations để phục vụ điều hướng (Navigation), kiểm tra sơ đồ ghế thực tế và truy vấn trạng thái chuyến xe thời gian thực.
- Không nhân bản các bảng cấu trúc phức tạp như Route, Schedule hay Vehicle sang mô-đun Booking.

**2. Snapshot dữ liệu vé cố định tại thời điểm đặt (Immutable Booking Snapshots):**

Để giữ nguyên lịch sử giao dịch khi danh mục thay đổi, Booking và Ticket lưu các snapshot cần thiết tại thời điểm giao dịch:

- **paid_fare_vnd:** Snapshot giá vé thực tế hành khách đã trả (không bị ảnh hưởng nếu Admin sau này điều chỉnh LegFare trên Trip).
- **seat_code_snapshot:** Snapshot mã vị trí ghế (ví dụ: "A05") tại thời điểm xuất vé.
- **from_stop_name_snapshot & to_stop_name_snapshot:** Snapshot tên trạm lên/xuống và địa chỉ trạm tại thời điểm mua vé.
- **trip_departure_time_snapshot:** Snapshot thời gian khởi hành dự kiến ban đầu.

**Quy tắc bảo toàn lịch sử:**

Dù Admin sau này có deactivate Vehicle, sửa tên Stop, hay điều chỉnh bảng giá LegFare của tuyến đường, toàn bộ thông tin trên BOOKING và TICKET đã phát hành vẫn giữ nguyên giá trị Snapshot ban đầu, bảo đảm 100% tính toàn vẹn cho công tác đối soát, khiếu nại và hoàn tiền.

**3. Các ràng buộc nghiệp vụ (Bất biến dữ liệu từ PA2)**

- **Ràng buộc Xe (UC-A01):** Sức chứa (capacity) và sơ đồ (layoutTemplate) bị CẤM sửa đổi một khi xe đã được gán cho bất kỳ Trip nào. Xe đã dùng chỉ được deactivate, không được delete.
- **Ràng buộc Chuyến (UC-A03):** Nếu một Trip đã có phân bổ hợp lệ (Confirmed Booking hoặc Seat

Hold chưa hết hạn), Admin CẤM KHÔNG ĐƯỢC đổi Vehicle, đổi Route, đổi thời gian tại các trạm hay sửa giá vé. Việc validation này phải thực hiện lại ngay trước lúc save database.

- **Chống trùng lặp thời gian:** Một Vehicle không được có khoảng thời gian chạy trùng lặp giữa nhiều Trip khác nhau.

**4. Hợp đồng dữ liệu dùng chung**

- **BookingQueryPort cung cấp lookupForStaff(crite**ria,ctx): StaffBookingPage và getTripTicketCounts(tripIds,ctx): TripTicketCounts[]. BookingOperationsPort cung cấp validateTicketForCheckIn/checkInTicket(tripId,ticketReference,boardingContext,ctx), hasEffectiveAllocations(tripId,ctx), cancelForOperatorTripCancellation(tripId,reason,ctx).
- **RequesterContext do I**dentity server dựng: accountId,sessionId,role,authenticated; caller Staff/Admin phải được xác thực và có đúng quyền. Không dùng role/staffId từ client làm căn cứ.

**Điểm đã thống nhất:**

- Tiền tệ lưu kiểu số nguyên (bigint - VND).
- Thời gian hệ thống lưu định dạng ISO-8601 UTC (timestamp).
- **Thời hạn giữ chỗ (Hold):** Hết hạn sau 10 phút kể từ thời điểm khởi tạo thành công (thao tác refresh/retry không kéo dài thời gian hold).
- **Đóng đặt vé mới (Cutoff):** Đóng chức năng đặt vé mới từ 30 phút trước giờ khởi hành tại trạm đầu tuyến (firstStopDepartureTime).

### 4.4 Component: Booking & Payment

#### *1. Mục đích, phạm vi và ánh xạ các ca sử dụng (Use Case)*

Booking & Payment quản lý Booking, hold theo Segment, các lượt và giao dịch thanh toán, Ticket, RefundRequest và ReconciliationCase. P07 chỉ đọc, P08 tạo Pending/hold, P09 xác minh tiền/phát vé nếu đủ điều kiện, P10 hủy cả Booking, A04 chỉ đọc. Trip & Operations cung cấp Trip/PhysicalSeat; Identity & Access cung cấp Account/RequesterContext; AI Assistance chỉ dùng phần đọc được phép.

#### *2. Vị trí trong kiến trúc và trách nhiệm từng tầng*

Modular Monolith: Java 21/Spring Boot, PostgreSQL. Các lớp miền: Booking, BookingSeat, SeatAllocation, PaymentAttempt, PaymentTransaction, Ticket, RefundRequest, ReconciliationCase; các service điều phối theo ownership, không ghi chéo bảng.

- **Presentation: hiển thị** dữ liệu máy chủ; không tự kết luận còn ghế, thanh toán thành công hoặc quyền hoàn tiền.
- **Application: Booking**Service điều phối P08; PaymentService điều phối P09; CancellationService điều phối P10/S04; TicketLifecycleService cấp vé/check-in; AdminBookingQueryService cung cấp A04 chỉ đọc. Mọi thao tác kiểm tra RequesterContext, quyền và ownership.
- **Domain:** SeatAllocationService kiểm tra Segment; PaymentTransaction lưu chứng cứ tiền; ReconciliationService đối soát. TicketLifecycleService ghi checkedInAt/checkedInByStaffId và audit(ticketId,bookingReference,tripId,staffAccountId,checkedInAt) nguyên tử.
- **Persistence:** một nguồn có thẩm quyền cho Booking/Payment/Ticket/Refund. PostgreSQL, JPA/SQL, cùng DataSource/transaction manager; REQUIRED và READ COMMITTED; khóa. ERD logical không đồng nhất 1:1 với lớp service.
- **Integration:** PA4 thanh toán mock phải ghi nhãn mô phỏng; PA5 dự kiến VNPay Sandbox và xác minh ở server. Chỉ back-office bên ngoài cập nhật kết quả hoàn tiền thực tế, không có chức năng chuyển tiền trong UC-P10/UC-A04.

#### *3. Hợp đồng dữ liệu bắt buộc giữa các thành phần*

- Trip & Operations cung cấp Trip, PhysicalSeat, orderedStops, giờ khởi hành đầu tuyến/giờ lên tại Stop đã chọn và giá chặng. BookingQueryPort phục vụ các phép đọc được phép; BookingOperationsPort phục vụ validate/check-in, allocation guard và hủy chuyến. Sửa Trip phải kiểm tra allocation trong

cùng transaction.

- **Identity & Access: Requester**Context do server dựng gồm accountId, sessionId, role, authenticated. Không tin ownerId/role/staffId do browser gửi. Các caller đều kiểm tra quyền, kể cả khi đi qua interface nội bộ.
- **Web Presentation: nhận DTO** chuẩn, các trạng thái tách biệt và requestId/error.code. P08 đọc/kiểm tra contact từ Account, cho người dùng xác nhận họ tên/email/phone và lưu snapshot liên hệ được chốt; không sử dụng dữ liệu giả định từ client làm căn cứ quyền.
- **Staff dùng Query** cho lookup/count và Operations cho validate/check-in/allocation/hủy Trip. validate chỉ đọc; check-in sau xác nhận phải kiểm tra lại và ghi một audit gồm ticketId, bookingReference, tripId, staffAccountId, checkedInAt. Trip & Operations không ghi trực tiếp Ticket.
- **AI: chỉ gọi Boo**kingQueryPort.getOwnedBookingFacts qua adapter giới hạn; trả đúng sự thật về Booking thuộc Passenger hiện tại, không nhận dữ liệu credential/contact dư thừa và không gọi OperationsPort.
- **Admin F16: AdminBookingQueryService cung cấp dữ** liệu, Web Presentation hiển thị. Chỉ đọc danh sách/chi tiết/KPI; không sửa Booking, xác nhận tiền, check-in hoặc chuyển tiền hoàn.

### *4. Cơ chế giao tiếp*

- Các HTTP endpoint minh họa dùng HTTPS/JSON; quyền xác thực luôn từ RequesterContext. Port nội bộ không tự trở thành API công khai.
- Các module backend gọi nhau qua BookingQueryPort và BookingOperationsPort đã duyệt; triển khai bằng dịch vụ nội bộ và UnitOfWork đồng nhất thay vì nhiều bản ghi Ticket hoặc Seat độc lập.
- Các thao tác đồng thời dùng DB/server clock và transaction; ước lượng từ frontend không phải nguồn xác thực. Không dùng UNIQUE(tripId, seatId) để cấm sai việc dùng lại ghế trên các đoạn không giao nhau.
- PA5: PaymentGatewayPort xác minh chữ ký, mã tham chiếu, currency, amount, status và providerTransactionId trên server; sự kiện trùng không phát thêm vé. PA4: kết quả mock được ghi nhãn rõ.
- RefundRequest Pending là nghĩa vụ được ghi nhận, không phải tiền đã trả; back-office mới có quyền đưa Completed/Failed dựa trên chứng cứ thực tế.

### *5. DTO/API tối thiểu — contract thiết kế PA3, kiểm chứng khi triển khai PA4*

- **GET /api/v1/trips/{tripId}/seat-availability?fromStopId=&toSto**pId= — công khai, chỉ đọc trạng thái ghế theo Segment; không tạo hold. Trả FREE/UNAVAILABLE/UNKNOWN có căn cứ và expiresAt nếu có.
- **POST /api/v1/book**ings — Passenger; request: {tripId,fromStopId,toStopId,seats:[{physicalSeatId,travellerName}],confirmedContact:{fullName,email,phone},idempotencyKey}. Mỗi seatId chỉ được xuất hiện một lần; số ghế 1–6; contact đối chiếu Account và lưu snapshot. Response: {bookingId,status:'Pending',totalAmountVnd,createdAt,expiresAt,seatCount}. Không chứa Ticket.
- **POST /api/v1/bookings/{bookingId}/payment-atte**mpts — Passenger sở hữu; {idempotencyKey}; response {attemptId,attemptOrdinal,amountVnd,status,checkoutMode,expiresAt}. Bắt buộc dùng số tiền đã snapshot, tối đa ba lượt và chỉ một Pending.
- **POST /api/v1/payments/provider-notifi**cation — chỉ adapter/provider; kiểm chứng ở server, định danh duy nhất providerTransactionId; trả kết quả xử lý lặp idempotent. Không coi redirect từ browser là thanh toán đã xác minh.
- **POST /api/v1/bookings/{bookingId}/ca**ncel — Passenger sở hữu; response {bookingStatus:'Cancelled',refund:{status,requestedAmountVnd},reconciliationStatus}. Không chuyển tiền trong ứng dụng.
- GET /api/v1/admin/bookings?bookingReference=&tripId=&boardingDate=&bookingStatus=&paymentStat

us=&page=&pageSize= — Administrator. Danh sách/chi tiết/summary chỉ đọc. Lọc ANY PaymentAttempt; hiển thị latestAttemptStatus và matchedAttemptIds.

ID nghiệp vụ: UUID; session/conversation: opaque ID do server cấp. UTC ISO-8601; hiển thị Asia/Ho_Chi_Minh; VND nguyên. Lỗi {error:{code,message},requestId}: 401/403/404/409/422/503; ghế lặp 422 DUPLICATE_SEAT, tranh ghế 409 SEAT_CONFLICT. Đây là contract thiết kế, chưa phải kết quả triển khai.

## *6. BẤT BIẾN VÀ TRANSACTION LIÊN MÔ-ĐUN*

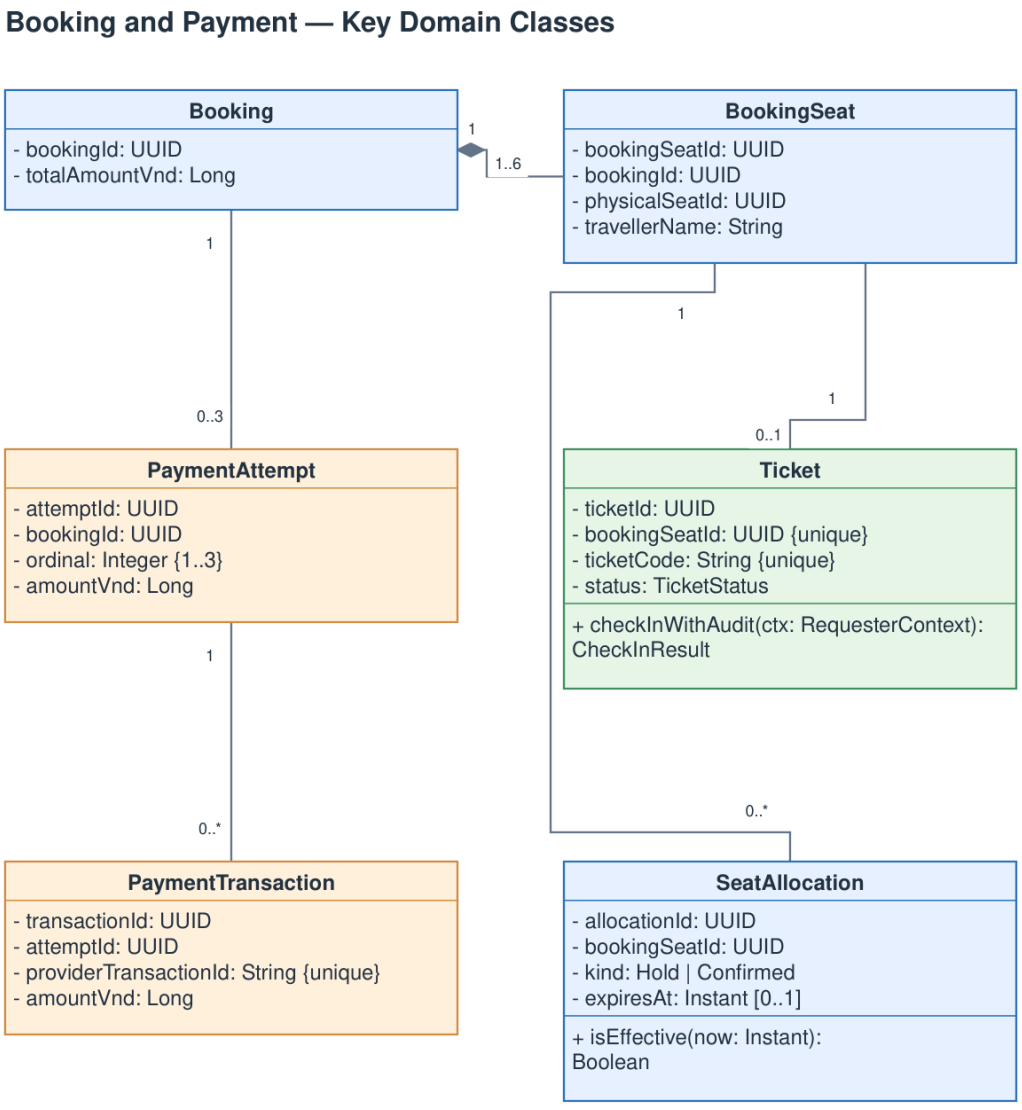
P07 trả ảnh trạng thái ghế tại thời điểm đọc; P08 kiểm tra lại nguyên tử. P09 vẫn ghi Payment Succeeded nếu chứng cứ tiền hợp lệ, kể cả hold đã hết hạn, nhưng KHÔNG chuyển Booking Expired/Cancelled thành Confirmed hoặc phát Ticket; lưu Reconciliation và nghĩa vụ hoàn toàn bộ khoản không khớp. P10 chỉ hủy cả Booking theo chính sách. S03 validate chỉ đọc, còn checkIn phải tái kiểm tra và lưu audit.

Khóa: Trip → Booking tăng dần → guard(tripId,physicalSeatId) tăng dần → Ticket/PaymentAttempt/PaymentTransaction → RefundRequest/ReconciliationCase. Áp dụng P08/A03/S04, P09/expiry/P10/S04, S03/P10; rollback toàn bộ và retry có giới hạn. Không gọi provider khi giữ khóa.

**Thiết kế lớp miền nghiệp vụ và cơ chế tích hợp**

**A. Lớp miền nghiệp vụ**

- **A1 — Booking:** Lưu bookingId, passengerAccountId, tripId, fromStopIndex, toStopIndex, status, totalAmountVnd, holdCreatedAt và expiresAt. Một Booking gồm một Trip, một Segment [from,to) và 1–6 BookingSeat; service điều phối các chuyển trạng thái.
- **A2 — BookingSeat:** Lưu bookingSeatId, bookingId, physicalSeatId, travellerName và fareAmountVnd. Tổ hợp (bookingId,physicalSeatId) duy nhất; Ticket tham chiếu bookingSeatId UNIQUE, không lưu thêm ticketId ngược ở BookingSeat.
- **A3 — SeatAllocation:** Lưu allocationId, bookingSeatId, tripId, physicalSeatId, Segment [fromStopIndex,toStopIndex), kind {Hold,Confirmed}, expiresAt? và releasedAt?. Mỗi BookingSeat có lịch sử allocation; chỉ phân bổ hiệu lực chặn ghế. PhysicalSeat thuộc Trip & Operations.
- **A4 — PaymentAttempt:** Lưu attemptId, bookingId, ordinal 1..3, frozenAmountVnd và status {Pending,Succeeded,Failed,Cancelled}. Mỗi Booking tối đa ba lượt thử và một lượt Pending tại một thời điểm.
- **A5 — PaymentTransaction:** Lưu transactionId, attemptId, providerTransactionId UNIQUE, amountVnd và verifiedAt. Mỗi bản ghi thể hiện một khoản tiền đã được provider xác minh, không phải một bản sao callback.
- **A6 — Ticket:** Lưu ticketId, bookingSeatId UNIQUE, ticketCode UNIQUE, status {Valid,CheckedIn,Cancelled}, issuedAt, checkedInAt? và checkedInByStaffId?. TicketLifecycleService phát vé/ghi check-in nguyên tử; Trip/Seat/Segment được truy qua BookingSeat → Booking.
- **A7 — RefundRequest:** Lưu refundId, bookingId, sourceTransactionId tham chiếu PaymentTransaction, reason, requestedAmountVnd, status {NotRequired,Pending,Completed,Failed} và idempotencyKey UNIQUE. Mỗi nghĩa vụ hoàn phải truy vết về khoản tiền gốc.
- **A8 — ReconciliationCase:** Lưu reconciliationId, sourceTransactionId, affectedAmountVnd, reason và status {NotRequired,ReviewRequired,Resolved}. Khoản tiền muộn/thừa có trường hợp đối soát riêng; tổng hoàn không vượt số đã nhận.
- **A9 — Tham chiếu dùng chung:** Account/RequesterContext thuộc Identity & Access; Trip/PhysicalSeat thuộc Trip & Operations. Lớp service không ánh xạ mặc định 1:1 với bảng ERD. Thiết kế chi tiết nằm trong PA3_Group11_BookingPayment_v2.1.drawio; Hình 12 là view lớp miền rút gọn.




**Hình 12.** Booking and Payment — Key Domain Classes *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

**B. Quan hệ, trạng thái và snapshot**

- **B1 — Cardinality:** Account 1—0..* Booking; Trip 1—0..* Booking; Booking 1—1..6 BookingSeat (composition); BookingSeat 1—0..1 Ticket; PhysicalSeat 1—0..* BookingSeat; BookingSeat 1—0..* SeatAllocation. Mỗi BookingSeat chỉ có một allocation hiệu lực tại một thời điểm.
- **B2 — Quan hệ thanh toán:** Booking 1—0..3 PaymentAttempt; PaymentAttempt 1—0..* PaymentTransaction; PaymentTransaction 1—0..* RefundRequest và 1—0..1 ReconciliationCase. CSDL ràng buộc nguồn tiền, khóa tham chiếu và idempotency.
- **B3 — Bộ trạng thái:** Booking: Pending/Confirmed/Expired/Cancelled; Payment: Pending/Succeeded/Failed/Cancelled; Ticket: Valid/CheckedIn/Cancelled; Refund: NotRequired/Pending/Completed/Failed; Reconciliation: NotRequired/ReviewRequired/Resolved. TripStatus dùng mô hình Trip & Operations.
- **B4 — Snapshot:** Booking lưu hành trình, Stop index/order, giá đã xác nhận và dữ liệu cần giữ cho lịch

sử giao dịch. Snapshot giá bất biến sau P08; lifecycle và audit cập nhật nhất quán trong transaction. Bảng con giữ FK, không tạo một Trip/Segment có thẩm quyền thứ hai.

**C. Segment và hiệu lực phân bổ**

- **C1 — Biểu diễn:** Segment dùng thứ tự Stop của Route/Trip thuộc Trip & Operations quản lý và khoảng nửa mở [fromStopIndex,toStopIndex), from < to. Không dùng ngày UTC để xác định ngày đi của Passenger.
- **C2 — Giao nhau:** Hai Segment trên cùng tripId/physicalSeatId giao nhau khi fromIndex < otherToIndex VÀ otherFromIndex < toIndex; A→B và B→C không giao nhau, A→C với B→C có giao nhau.
- **C3 — Hiệu lực:** Allocation Hold hiệu lực khi Booking=Pending, kind=Hold, chưa released và t < expiresAt. Allocation Confirmed hiệu lực khi Booking=Confirmed, kind=Confirmed, chưa released. Expired/Cancelled/released không chặn ghế; khác tripId không xung đột.
- **C4 — Ngày đi và cutoff:** Theo BR-P04-05, travelDate của Passenger và AI là ngày địa phương Việt Nam tại Stop LÊN XE đã chọn, kể cả chuyến qua đêm; cutoff đặt chỗ vẫn 30 phút trước giờ rời Stop ĐẦU TUYẾN.

**D. Tạo Booking nguyên tử và idempotency**

- **D1 —** Kiểm tra trước transaction: đăng nhập Passenger, Trip Scheduled, Stop từ trước đến sau, đủ 1–6 ghế, seatId không trùng, mỗi ghế đúng một travellerName, thông tin liên hệ Account hợp lệ và hạn bán. Ghế trùng trả lỗi 422/đề nghị sửa; KHÔNG âm thầm tự loại rồi đặt số ghế ít hơn.
- **D2 —** Mở UnitOfWork và khóa Trip → Booking nếu có → guard từng (tripId,physicalSeatId) tăng dần, kiểm tra trạng thái Trip/hạn đặt/giá và mọi allocation còn hiệu lực giao Segment bên trong transaction. A03/S04 phải dùng cùng cơ chế; không chỉ kiểm tra rồi ghi ở hai transaction tách biệt.
- **D3 —** Nếu có một ghế xung đột thì rollback toàn bộ 1–6 ghế. Không dùng UNIQUE(tripId,seatId) cứng vì sẽ cấm ghế tái sử dụng trên Segment kề nhau. Khi DB lỗi/deadlock, rollback và báo thử lại có giới hạn.
- **D4 —** Khi mọi điều kiện hợp lệ, tạo Booking Pending, BookingSeat, SeatAllocation Hold và snapshot tiền/người đi/Stop cùng một đơn vị giao dịch. holdCreatedAt được lấy từ đồng hồ DB/máy chủ sát thời điểm ghi thành công, KHÔNG lấy lúc vừa nhận request trước khi đợi khóa. expiresAt=holdCreatedAt+10 phút, chung cho tất cả ghế; commit xong mới báo thành công.
- **D5 —** Nếu commit kéo dài, hệ thống vẫn so sánh expiresAt với thời gian có thẩm quyền khi thanh toán; không tự cộng thêm 10 phút từ lúc gửi response. Idempotency key gắn (Passenger, action, requestHash); gửi lại sau mất response trả cùng bookingId/expiresAt khi có bản ghi đã commit, hoặc từ chối theo contract. Không tạo một hold mới để gia hạn yêu cầu cũ.
- **D6 —** Tác vụ dọn Expired có thể chạy trễ nhưng truy vấn availability/check-in/payment luôn dùng expiresAt thực; không phụ thuộc lịch chạy cleanup.

**E. Thanh toán, callback, check-in và transaction**

- **E1 — Tạo lượt thanh toán:** P09 tạo PaymentAttempt Pending dưới khóa Booking, kiểm tra owner, hạn hold và giới hạn tối đa ba lượt/một Pending. PaymentGatewayPort xác minh nguồn, chữ ký, transactionId, mã tham chiếu, amount/currency trên server; deduplicate callback theo event và providerTransactionId UNIQUE. Mỗi khoản Succeeded độc lập có PaymentTransaction riêng.
- **E2 — Xử lý tiền và callback:** Chỉ khoản Succeeded đầu tiên còn đủ điều kiện (Booking Pending, hold chưa hết, phân bổ hợp lệ) mới được chuyển Booking→Confirmed, hold→Confirmed và cấp đúng 1 Ticket cho mỗi BookingSeat trong một transaction. Mọi callback lặp là no-op. Nếu Succeeded nhưng hold hết hạn/Booking Cancelled/ghế xung đột hoặc khoản tiền thừa, giữ PaymentTransaction Succeeded, mở/tái sử dụng ReconciliationCase riêng và RefundRequest hoàn TOÀN BỘ khoản đó; KHÔNG phát vé hoặc hồi sinh Booking.
- **E3 — Check-in:** Chỉ Staff được check-in. validateTicketForCheckIn chỉ đọc; sau xác nhận, checkInTicket kiểm tra lại Trip Scheduled/Departed hợp lệ, Ticket Valid, Booking Confirmed, Stop lên và quyền Staff trong transaction. Ghi checkedInAt/checkedInByStaffId và audit đúng một lần; retry

không tạo audit mới.

- **E4 — Hủy chuyến:** S04 dùng cùng UnitOfWork cho Trip, Booking, Ticket, allocation và nghĩa vụ hoàn tiền. Thất bại rollback toàn bộ, không trả thành công một phần.

**F. Hủy Booking và tính tiền hoàn**

- **F1 — Điều kiện hủy:** P10 hủy toàn bộ Booking, không hủy riêng ghế. Pending được hủy không hoàn tiền; Confirmed chỉ hủy khi còn ít nhất 12 giờ trước giờ lên dự kiến tại Stop đã chọn và không Ticket nào CheckedIn.
- **F2 — Hoàn do hành khách hủy:** Theo BR-P10-07, F là tổng vé đã trả của cả Booking (VND nguyên không âm): refundVND = floor((9 × F + 5) / 10); phí = F − refundVND. Làm tròn half-up một lần cho cả Booking. Ví dụ F = 100.005 → hoàn 90.005, phí 10.000.
- **F3 — Hoàn do nhà xe hoặc tiền không khớp:** S04 nhà xe hủy Trip hoàn 100%; khoản tiền thừa/muộn hoàn đủ khoản gốc cần đối soát. Mỗi nghĩa vụ hoàn gắn sourceTransactionId và không vượt tiền thực nhận.
- **F4 — Trạng thái hoàn:** Refund Pending là nghĩa vụ đã ghi nhận, không phải tiền đã chuyển. Back-office cập nhật Completed/Failed theo bằng chứng thực tế; sourceTransactionId và idempotencyKey ngăn hoàn trùng.

### *Thiết kế UC-A04 — Review Bookings and Operations Overview (chỉ đọc)*

**UC-A04 (F16) — READ MODEL, BỘ LỌC VÀ KPI**

- **Administ**rator cần phiên hợp lệ và quyền đọc trong phạm vi nhà xe; mọi GET kiểm tra RequesterContext. Booking & Payment sở hữu AdminBookingQueryService, Web Presentation hiển thị. A04 không tạo/sửa Booking, check-in, duyệt Payment hoặc chuyển tiền hoàn.
- **Nguồn dữ liệu:** Booking, BookingSeat, Ticket, PaymentAttempt, PaymentTransaction, RefundRequest, ReconciliationCase cùng Trip/PhysicalSeat từ Trip & Operations và thông tin liên hệ vận hành tối thiểu từ Account của Identity & Access. Query read model danh sách/chi tiết/KPI tách riêng; tổng hợp PaymentTransaction theo transactionId trước khi JOIN nhiều Ticket để không nhân tiền.
- **A04 lọc bookingReference,tr**ipId,boardingDate,bookingStatus,paymentStatus. boardingDate là travelDate trong BR-A04: ngày tại Stop lên, giờ Việt Nam. Query parameterized và kiểm tra quyền.
- **paymentStatus khớp ANY PaymentAtt**empt; mỗi dòng trả latestAttemptStatus và matchedAttemptIds. Booking có Failed rồi Succeeded vẫn khớp lọc Failed nhưng hiển thị trạng thái gần nhất là Succeeded.
- **KPI và danh sách áp c**ùng predicate với nhãn "Theo bộ lọc hiện tại". Sắp boardingDate desc, bookingId desc; page=1, pageSize=20, tối đa 100. Các truy vấn trong một response dùng snapshot đọc nhất quán.
- **KPI đúng BR-A04-03/04:** (1) số Booking phân biệt; (2) Ticket Valid, CheckedIn, Cancelled riêng và Valid+CheckedIn là vé hiện được tính phát hành; (3) tổng tiền provider-verified Succeeded của PaymentTransaction phân biệt, gồm tiền muộn/thừa — ghi "Tiền đã ghi nhận", KHÔNG gọi doanh thu; (4) số giao dịch có Reconciliation ReviewRequired. PA4 hiển thị riêng tiền mock có nhãn MÔ PHỎNG.
- **DTO danh sách:** {bookingId,bookingReference,tripId,boardingStopId,alightingStopId,boardingDate,seatCount,booking Status,latestAttemptStatus,matchedAttemptIds,reconciliationStatus,refundStatus}. Chi tiết trả traveller/seat, Ticket, PaymentAttempt/Transaction, RefundRequest và contact tối thiểu theo quyền; không xuất bí mật.
- **Ví dụ minh họa:** một Booking có 3 Ticket và 2 lần thử (Failed/Succeeded) không được nhân tổng tiền lên 6 lần; 100.000 VND thanh toán hợp lệ + 100.000 VND thừa = 200.000 VND tiền đã ghi nhận, chỉ phát Ticket cho phần hợp lệ, khoản thừa cần đối soát.
- **Lỗi:** 401 khi thiếu phiên, 403 sai quyền, 404 khi tài nguyên không được phép thấy, 422 bộ lọc ngày

sai, 503 không truy xuất được. Trạng thái rỗng hiển thị 0, không dựng số liệu giả; refresh không được khẳng định giá trị cache lỗi thời là số tiền hiện tại. Đây là thiết kế phục vụ TC-51/TC-52, chưa có kiểm thử chạy.

## 4.5 Component: AI Assistance

### *1. Trách nhiệm và use case đáp ứng*

AI Assistance cung cấp gợi ý chuyến (UC-P11, F03) và trả lời hỗ trợ khách hàng (UC-P12, F10) cho Passenger. Gemini Developer API là External AI Service hỗ trợ hai UC này; Passenger vẫn là actor chính. Thành phần chuyển câu tiếng Việt thành tiêu chí đã kiểm tra hoặc loại câu hỏi, lấy dữ liệu thật qua module sở hữu và trả câu trả lời có nguồn hoặc phương án dự phòng. Thành phần không sở hữu dữ liệu Trip, Booking, tài khoản hay nội dung chính sách, và không tạo, sửa hoặc hủy Booking, Seat Hold, Payment, Ticket, Refund.

### *2. Kết nối và cách giao tiếp*

- Web Presentation → AI Assistance: HTTPS/JSON tới AiAssistantApi, cùng origin với frontend; lỗi theo dạng {error:{code,message},requestId}.
- AI Assistance → Identity & Access: lời gọi nội bộ; nhận RequesterContext (accountId, sessionId, role, authenticated) do server dựng; khách chưa đăng nhập dùng AnonymousSession do Identity cấp.
- AI Assistance → Trip & Operations: chỉ đọc qua TripQueryPort của Trip & Operations.
- AI Assistance → Booking & Payment: chỉ đọc qua BookingQueryPort.getOwnedBookingFacts. AI không được gọi Staff lookup/count hoặc bất kỳ thao tác nào của BookingOperationsPort.
- GeminiGateway → Gemini Developer API: HTTPS qua Internet, gọi từ backend.

### *3. Quy tắc dữ liệu và giới hạn hành động*

- Đầu ra của AI chỉ là tiêu chí chưa xác minh; kết quả Trip luôn do ứng dụng lấy từ TripQueryPort, tối đa 5 chuyến, sắp theo giờ lên tăng dần; AI không tự tính giá hay chỗ trống và không tạo hold (BR-P11-02 đến BR-P11-04).
- Ngày trong tiêu chí là ngày tại Stop lên theo giờ Việt Nam; mốc đóng bán tính từ Stop đầu tuyến (BR-P11-01, BR-P04-05).
- Nhánh Booking: Identity xác thực người gọi; BookingQueryPort truy xuất và kiểm tra Passenger sở hữu Booking. Không tin accountId/role từ browser. Ứng dụng hiển thị BookingFacts trực tiếp, không đưa chúng vào prompt (BR-P12-05).
- BookingFacts gồm bookingReference, journey, seats [1..6], tickets [0..6], bookingStatus, paymentSummary, refundStatus, reconciliationStatus; không có tên hành khách, liên hệ, credential hay bí mật cổng thanh toán. Booking Pending hoặc Expired có thể có 0 Ticket.
- Gemini Free Tier có thể dùng input/output để cải thiện sản phẩm theo điều khoản Google; demo chỉ dùng dữ liệu giả và tiêu chí/FAQ công khai đã kiểm tra. Không gửi BookingFacts, traveller/contact, bí mật hoặc raw text không xử lý an toàn. UNSAFE_INPUT chuyển sang form có cấu trúc. Không khẳng định provider không lưu dữ liệu.

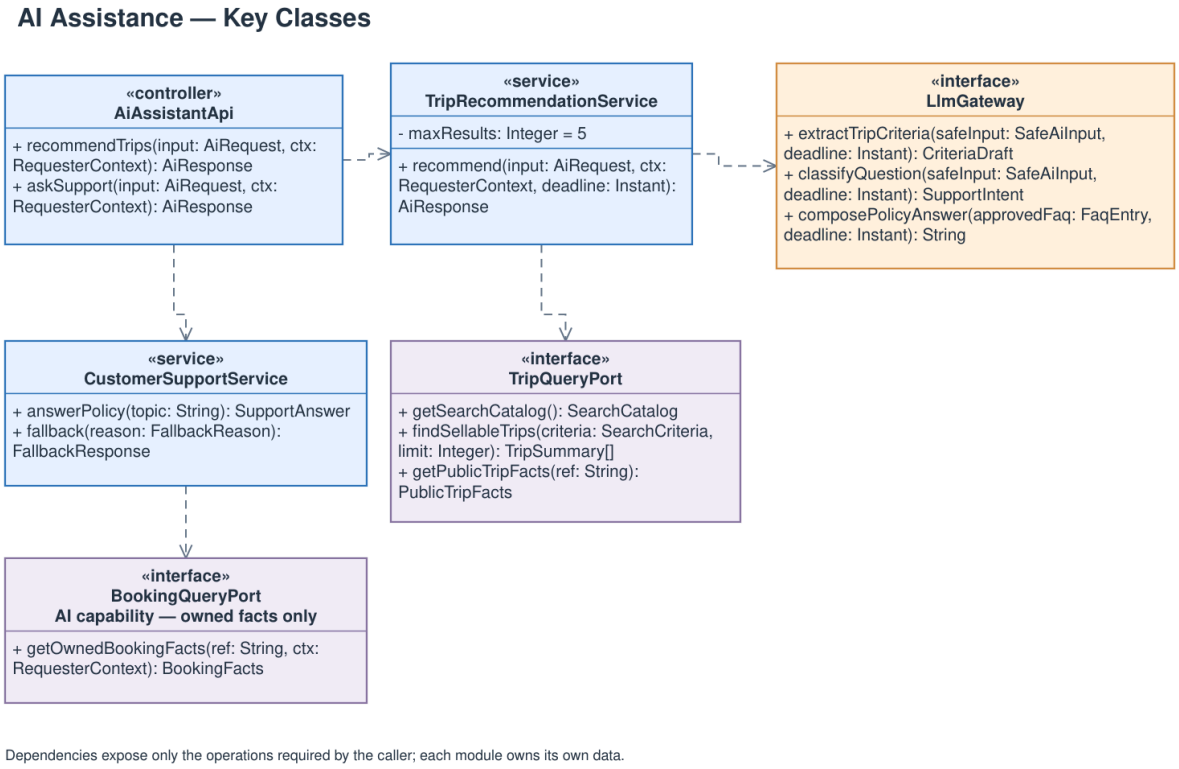
### *4. Hạn mức, thời hạn và phản hồi*

- Quota ứng dụng P11/P12: 5 yêu cầu/60 giây trượt mỗi account/phiên ẩn danh; 200/ngày Asia/Ho_Chi_Minh toàn demo; cấp nguyên tử, chỉ lưu metadata. Đây là trần ứng dụng, không phải quota Google. Provider có RPM/TPM/RPD riêng theo project/model; giới hạn thực tế kiểm tra AI Studio trước demo. Hết quota trả fallback, không tự gọi dịch vụ trả phí.
- Thời hạn 10 giây tính cho toàn bộ yêu cầu; không tự retry nhà cung cấp quá thời hạn này.
- AiResponse có requestId, status (Success | Clarification | Fallback), sourceType, sourceRef. NO_RESULT là kết quả rỗng có nguồn, không phải lỗi dịch vụ. Vượt hạn mức trả 429 kèm retryAfterSeconds; AI lỗi hoặc quá hạn trả 503 kèm FallbackResponse và UC thay thế (UC-P04 hoặc UC-P06); Booking ngoài quyền trả 404, không xác nhận Booking có tồn tại.

- FAQ là bundle có phiên bản, gồm đặt vé, hold, thanh toán, hủy/hoàn tiền và check-in; mỗi mục có sourceRef về UC/BR và approvalStatus. FaqRepository chỉ trả mục Approved. Contact của prototype để trống; thiếu FAQ thì báo chưa có chính sách được phép dùng, không tự tạo kênh liên hệ.

### *5. Sơ đồ lớp và giải thích các lớp chính*

Nguồn: PA3_Group11_AIAssistance_v2.3.drawio, gồm 3 view: Orchestration and Read Contracts, Gateway Policy and Quota, Response Data. View lớp chính trong SAD lấy từ PA3_Group11_SADViews_v2.3.drawio; trong view này BookingQueryPort chỉ hiện capability getOwnedBookingFacts mà AI được dùng. Màu: xanh dương — lớp AI; cam — gateway/adapter; tím — interface và DTO tham chiếu module khác; xám — dịch vụ bên ngoài.




**Hình 13.** AI Assistance — Key Classes *(nguồn: PA3_Group11_SADViews_v2.3.drawio)*

| Lớp | Loại / chủ sở hữu | Thuộc tính và phương thức chính | Vai trò |
| --- | --- | --- | --- |
| AiAssistantApi | controller | recommendTrips(input: AiRequest, ctx: RequesterContext): AiResponse; askSupport(input: AiRequest, ctx): AiResponse | Điểm vào HTTPS duy nhất; nhận RequesterContext từ Identity rồi chuyển cho service |
| TripRecommendationService | service | maxResults = 5; recommend(input, ctx, deadline: Instant): AiResponse | Điều phối UC-P11: hạn mức → kiểm tra input → AI trích tiêu chí → TripCriteriaValidator → TripQueryPort |

| Lớp | Loại / chủ sở hữu | Thuộc tính và phương thức chính | Vai trò |
| --- | --- | --- | --- |
| CustomerSupportService | service | answer(input, ctx, deadline): AiResponse; answerPolicy(topic): SupportAnswer; answerOwnedBooking(ref, ctx): SupportAnswer; fallback(reason): FallbackResponse | Điều phối UC-P12 theo SupportIntent; nhánh Booking chỉ gọi BookingQueryPort.getOwnedBookingFacts |
| InputSanitizer | service | sanitizeOrReject(text): SafeInputResult | Kiểm tra allowlist; trả SafeAiInput hoặc từ chối với UNSAFE_INPUT |
| AiRequestLimiter / AiQuotaRepository | service + repository | perMinuteLimit = 5, dailyDemoLimit = 200; tryAcquire(principal: PrincipalKey, now): QuotaResult; acquireBothLimitsAtomically(principal, day: LocalDate, now): QuotaResult | Hạn mức dùng chung P11/P12, cấp nguyên tử cả hai giới hạn |
| LlmGateway «interface» / GeminiGateway «adapter» | AI Assistance | extractTripCriteria(safeInput, deadline): CriteriaDraft; classifyQuestion(safeInput, deadline): SupportIntent; composePolicyAnswer(approvedFaq, deadline): String; model và apiKeyRef từ môi trường | Tách nghiệp vụ khỏi nhà cung cấp; chỉ GeminiGateway gọi Gemini Developer API |
| TripCriteriaValidator | service | validate(draft: TripCriteriaDraft, catalog: SearchCatalog): CriteriaResult | Đối chiếu bản nháp với Stop, ngày, loại xe thật; thiếu hoặc sai thì hỏi lại |
| FaqRepository / FaqEntry | repository + value object | approvedBundleVersion; findApproved(topic): FaqEntry [0..1]; topic, content, version, sourceRef, approvalStatus | FaqRepository chỉ trả nội dung có approvalStatus=Approved và nguồn chính sách rõ ràng. |
| SupportContactConfig | configuration | channel: String [0..1]; getConfiguredContact(): String [0..1] | Kênh liên hệ chính thức từ cấu hình; để trống khi chưa có |
| AiResponse «abstract DTO» và TripRecommendationResult, SupportAnswer, FallbackResponse | DTO | requestId, status, sourceType, sourceRef; trips [0..5], criteria; answer, bookingFacts [0..1], publicTripFacts [0..1]; reason, suggestedUseCase, retryAfterSeconds | Dữ liệu phản hồi dùng cho giao diện AI trong Web Presentation. |
| FallbackReason «enumeration» | AI Assistance | TIMEOUT, LIMIT_REACHED, LOGIN_REQUIRED, BOOKING_INACCESSIBLE, NO_RESULT, INVALID_CRITERIA, UNSUPPORTED_LANGUAGE, UNSAFE_INPUT, ACTION_NOT_ALLOWED, POLICY_UNKNOWN, SERVICE_UNAVAILABLE | Ánh xạ các luồng thay thế của UC-P11 và UC-P12 |
| TripQueryPort «interface» | Trip & Operations | getSearchCatalog(): SearchCatalog; findSellableTrips(criteria: SearchCriteria, limit): TripSummary[]; getPublicTripFacts(ref): PublicTripFacts | Contract chỉ đọc do Trip & Operations cung cấp |
| BookingQueryPort «interface» (capability hạn chế) | Booking & Payment | getOwnedBookingFacts(ref, ctx: RequesterContext): BookingFacts | Capability đọc Booking riêng tư được cấp cho AI sau kiểm tra ownership |

| Lớp | Loại / chủ sở hữu | Thuộc tính và phương thức chính | Vai trò |
| --- | --- | --- | --- |
| RequesterContext | Identity & Access | accountId/sessionId/role có thể vắng khi authenticated=false; anonymousConversationId? do server quản lý; không chứng minh ownership | Danh tính người gọi do Identity dựng phía server |
| Gemini Developer API «external service» | Bên ngoài | — | External AI Service; không phải lớp ứng dụng |

**Quan hệ chính:** AiAssistantApi phụ thuộc TripRecommendationService, CustomerSupportService và AiRequestLimiter; hai service phụ thuộc InputSanitizer và LlmGateway; GeminiGateway hiện thực LlmGateway và gọi Gemini Developer API qua HTTPS; TripRecommendationService dùng TripCriteriaValidator và TripQueryPort; CustomerSupportService dùng FaqRepository, SupportContactConfig, TripQueryPort và BookingQueryPort; FaqRepository 1 — 0..* FaqEntry; TripRecommendationResult, SupportAnswer, FallbackResponse kế thừa AiResponse.

## 5. Deployment

## 6. Implementation View